DevOps Question Bank
AWS

81 real interview questions - de-duplicated, topic-wise, Basic / Advanced / Scenario-Based

Every answer: plain-English idea · technical answer · example · what to say in the interview

By Priyanka Gupta — 25 September 2026

How to use this question bank

Each question has four parts:

In simple words (green box) - a plain-English analogy. Read this first; if you understand it, the technical part becomes easy to remember.

Detailed answer + Example - the real technical answer, with commands / YAML / code you can show.

What to say to the interviewer (blue box) - 2-3 lines to say out loud. Practise these aloud.

Questions that were asked more than once across the lists were merged into one (the other wordings are shown in grey as 'Also asked as'). Language-specific (Python/PHP) and non-DevOps questions were removed. Where a pasted 'model answer' was wrong, the corrected version is given.

Suggested order: all Basic questions first, then Scenario-Based (most interviews are troubleshooting-heavy), then Advanced. Use the PDF bookmarks panel to jump to any question.

Topic	Basic	Advanced	Scenario	Total
AWS	18	36	27	81

Questions in this topic

Basic 6

Q1. Which AWS services have you worked on, and which are you most proficient in? . . . 6

Q2. Explain the main networking concepts in AWS. . . . 6

Q3. What is an AWS VPC and a subnet? . . . 7

Q4. What is the purpose of a NAT Gateway? . . . 8

Q5. What is the difference between a Security Group and a NACL? . . . 9

Q6. What is an IAM role? . . . 9

Q7. What are the different S3 storage classes and their use cases? . . . 10

Q8. What is an S3 lifecycle policy? . . . 10

Q9. What is CloudFront, and when would you use it? . . . 12

Q10. What is Route 53, and how is it different from CoreDNS? . . . 13

Q11. What is a read replica in RDS? . . . 13

Q12. Explain the architecture of Amazon ECS. . . . 14

Q13. Why use Amazon EKS instead of ECS? What are the key differences? . . . 14

Q14. Which services can be integrated with a CDN (CloudFront)? . . . 15

Q15. What is the default port for DynamoDB? . . . 16

Q16. How do you add storage to an existing EC2 instance? . . . 16

Q17. What is the difference between an ALB and an NLB, and when would you prefer an NLB, especially with databases? . . . 17

Q18. What is AWS KMS used for, and what should be stored in it? . . . 19

Advanced 20

Q19. How do you allow one AWS account to safely access resources in another account (cross-account access)? . . . 20

Q20. How would you access data in an S3 bucket in Account A from an application running on EC2 in Account B? What permissions are needed on the bucket side? . . . 20

Q21. How do you secure S3 buckets in a multi-account setup? . . . 21

Q22. How can you access an EC2 instance that does not have a public IP? . . . 22

Q23. An EC2 instance in a private subnet must download packages without a NAT Gateway or bastion host. How? . . . 23

Q24. What is VPC peering and how does it work? If VPC A is peered with B, and B with C, can A talk to C? .

Q25. How can Instance 2 (with a static IP) communicate with Instance 1, which is in a private subnet behind a multi-AZ load balancer? . . . 25

Q26. Your application is hosted in S3 and users are in different countries. How do you reduce latency? . 25

Q27. You want to serve users in different countries. How do you route traffic based on user location? . . 26

Q28. If the primary RDS database fails, how can you promote a read replica? . . . 26

Q29. How do you handle database failover in RDS Multi-AZ? . . . 27
Q30. How can a developer securely access a private RDS instance without a bastion host? . . . 28
Q31. How do you configure priority-based Auto Scaling? . . . 28
Q32. How do you automate ECS and EKS node scaling? . . . 29
Q33. How would you perform a zero-downtime deployment in AWS? . . . 30
Q34. How do you implement API Gateway with Lambda for microservices? . . . 31
Q35. How do you handle rate limiting and throttling in API Gateway? . . . 31
Q36. What are best practices for AWS serverless services (Lambda, API Gateway, DynamoDB)? . . . 32
Q37. How would you design separate environments (Dev, Test, Prod) in AWS? . . . 33
Q38. What are best practices for VPC design at enterprise scale? . . . 33
Q39. How do you design a highly available architecture across Multi-AZ and Multi-Region? . . . 34
Q40. What is your strategy for Disaster Recovery and Backup in AWS? . . . 35
Q41. How do you handle secret rotation using Secrets Manager or Parameter Store? . . . 36
Q42. What is your approach for centralized secrets management for 100+ microservices? . . . 37
Q43. What strategy do you use to rotate access keys automatically? . . . 38
Q44. What metrics do you monitor for EC2, ECS, and RDS using CloudWatch? . . . 38
Q45. How do you optimize AWS cost for EC2, NAT, and S3? . . . 40
Q46. What are best practices to protect an AWS-hosted web application from attacks? . . . 40
Q47. Can you write a custom IAM policy? Explain its structure and how AWS evaluates policies. . . . 41
Q48. Two employees work different shifts (10 AM-5 PM and 6 PM-2 AM). How do you give them AWS access based on time? . . . 42
Q49. How do you connect AWS to an on-premises data center? . . . 43
Q50. How do you manage and connect to services like EC2, RDS, EKS and ECS? . . . 44
Q51. How do you establish database connections from your applications in deployments? . . . 45
Q52. How do you create AWS Lambda functions and manage their deployment artifacts? . . . 46
Q53. What is the hub-and-spoke network model? . . . 47
Q54. If you do not want to use Auto Scaling policies, how else can you scale EC2? . . . 48

Scenario-Based **48**

Q55. An application behind an ALB is returning 503 errors. How would you troubleshoot it? . . . 48
Q56. An EC2 instance behind an ALB becomes unreachable. How do you troubleshoot it in an existing infrastructure, and which metrics, tools and CLI commands do you use? . . . 49
Q57. What is the typical latency of a load balancer, and how do you troubleshoot high latency on an ALB? 51
Q58. Users are reporting slow application performance. Which AWS services and metrics would you check first? . . . 52
Q59. Two users access the same application: one sees low latency and the other sees high latency. How do you troubleshoot? . . . 53
Q60. An Auto Scaling Group is not launching new instances during traffic spikes. How would you investigate? . . . 54

Q61. Your EC2 instances scale up but never scale down. How would you debug and fix it? 54
Q62. An RDS database becomes unavailable during business hours. What is your recovery approach? . 55
Q63. An RDS instance is hitting max connections during peak traffic. How would you handle it? 56
Q64. Your ECS service is stuck in PROVISIONING. What steps would you take? 57
Q65. Describe an incident where an EC2 instance crashed or became unresponsive. What did you do? 58
Q66. An S3 bucket used for artifact storage was accidentally made public. What is your fix? 59
Q67. NAT Gateway costs doubled in 24 hours during a live event, with no infrastructure changes. What could be causing it? 60
Q68. Your team sees a spike in cloud costs. How do you find out why and cut costs without hurting performance? 61
Q69. How would you handle a sudden traffic spike from 10K to 200K users? 62
Q70. How do you design a fault-tolerant system that keeps running when one Availability Zone is down? 63
Q71. You are asked to migrate workloads from on-premises to AWS with zero downtime. How would you approach it? 64
Q72. What strategies do you use for database migration with zero downtime? 65
Q73. You must ship multi-region failover for live events in 2 weeks, and DNS-based routing is not allowed. What is your plan? 66
Q74. Application latency suddenly increased. The app runs in a private subnet behind a NAT Gateway. How do you investigate? 66
Q75. How would you expose a backend API securely through CloudFront and WAF, allowing only a certain IP range? 67
Q76. CloudWatch alerts are firing for Lambda timeout errors. How do you debug? 68
Q77. You want to auto-remediate high CPU on EC2 by rebooting it through Lambda. What is the logic? 69
Q78. An automation job that starts and stops EMR clusters suddenly fails with IAM errors. How do you debug and fix it? 70
Q79. An EMR cluster runs 24x7 but is used only 4 hours a day. What automation do you implement? . . 71
Q80. How do you identify underutilized EC2 instances across multiple AWS accounts? 72
Q81. Some APIs behind API Gateway are unsecured. How would you secure them? 73

AWS

Basic

Q1. Which AWS services have you worked on, and which are you most proficient in?

Also asked as: What AWS services are you proficient in? How many AWS services have you used? Which AWS services have you used in production?

> **In simple words:** The interviewer is not asking for a shopping list. They want to hear which services you have actually run in production and what you did with each one.

Detailed answer

Group your answer by category and attach a real task to each service. Five services you truly understand beat twenty you only read about.

- **Compute:** EC2, EKS, Lambda, Auto Scaling Groups
- **Networking:** VPC, subnets, Security Groups, ALB/NLB, Route 53, VPC endpoints
- **Data:** RDS (PostgreSQL/MySQL), ElastiCache (Redis), MSK (Kafka), S3
- **Operations:** CloudWatch (metrics, alarms, logs), EventBridge, SNS, SES, Systems Manager
- **Security:** IAM roles and policies, Secrets Manager, KMS

Example:

```
"I work across about 15 services daily. EKS runs our apps, RDS/ElastiCache/MSK are the
data layer, CloudWatch + EventBridge + Lambda power our alerting and daily health
reports to Teams, and IAM roles give every workload least-privilege access."
```

> ***What to say to the interviewer:*** *I have hands-on experience with roughly 15 AWS services, mainly EC2, EKS, VPC, RDS, ElastiCache, MSK, S3, IAM, CloudWatch, Lambda and EventBridge. For example, I built Lambda functions triggered by EventBridge that collect EKS, RDS, Redis and Kafka health and post a daily report to Teams, using an IAM execution role rather than stored keys.*

Q2. Explain the main networking concepts in AWS.

> **In simple words:** Think of AWS networking as a gated housing society: the society wall (VPC), blocks (subnets), road signs (route tables), the main gate (Internet Gateway), a one-way exit (NAT), guards (SG/NACL), private tunnels to AWS shops (endpoints), bridges to other societies (peering/TGW), the address book (Route 53) and the reception desk (load balancer).

Detailed answer

Concept	What it is	Everyday analogy
VPC	Isolated private network in a region with a CIDR range	The society wall
Subnet	Slice of the VPC in one AZ; public or private	A block inside the society
Route table	Rules for where traffic goes next	Road signs
Internet Gateway (IGW)	Two-way internet door for public subnets	Main gate
NAT Gateway	Outbound-only internet for private subnets	Exit-only gate
Security Group	Stateful allow-list on each instance/ENI	Guard at each flat
NACL	Stateless allow/deny rules per subnet	Guard at each block
VPC Endpoint	Private path to AWS services (gateway: S3/DynamoDB; interface: most others)	Private tunnel to shops
Peering / Transit Gateway	Connect VPCs 1-to-1 / hub-and-spoke	Bridge / central highway
VPN / Direct Connect	Connect on-premises to AWS	Private road to the office
Route 53	Managed DNS (public and private zones)	Address book
Elastic Load Balancer	ALB (HTTP L7), NLB (TCP/UDP L4), GWLB (appliances)	Reception desk
Elastic IP	Static public IPv4 address	Permanent phone number

Example:

```
Internet -> IGW -> ALB (public subnets) -> App (private subnets, SG from ALB)
                                             |-> NAT GW -> internet (outbound only)
                                             |-> S3 gateway endpoint (private)
                                            +-> RDS (data subnets, SG from app)
```

> ***What to say to the interviewer:*** *A VPC is the private network, divided into public and private subnets per AZ, with route tables sending traffic to an Internet Gateway, NAT Gateway, endpoints, peering or Transit Gateway. Security Groups and NACLs control access, Route 53 handles DNS and load balancers spread traffic across targets.*

Q3. What is an AWS VPC and a subnet?

> **In simple words:** A VPC is your own private, fenced plot of land inside AWS. Subnets are the rooms you divide it into: some rooms have a door to the street (public), some do not (private).

Detailed answer

- **VPC (Virtual Private Cloud):** an isolated virtual network in one region, defined by a CIDR block such as `10.0.0.0/16` (65,536 IPs).

- **Subnet:** a slice of the VPC CIDR that lives in exactly **one Availability Zone**, for example `10.0.1.0/24`.
- **Public subnet:** its route table has a route `0.0.0.0/0 -> Internet Gateway`.
- **Private subnet:** no route to the IGW; outbound internet (if needed) goes through a NAT Gateway.
- AWS reserves 5 IPs in every subnet (network, router, DNS, future use, broadcast).

Example:

```
VPC 10.0.0.0/16 (ap-south-1)
 +-- public-a   10.0.1.0/24  (AZ a)  -> IGW       : ALB, NAT GW
 +-- private-a  10.0.11.0/24 (AZ a)  -> NAT GW    : EKS nodes / app servers
 +-- data-a     10.0.21.0/24 (AZ a)  -> no internet : RDS, Redis
```

> ***What to say to the interviewer:*** *A VPC is a logically isolated network in a region with its own IP range. I split it into subnets per AZ: public subnets route to an Internet Gateway for load balancers, and private subnets hold apps and databases, reaching the internet only through a NAT Gateway if needed.*

Q4. What is the purpose of a NAT Gateway?

> **In simple words:** A NAT Gateway is a one-way door for a private room: people inside can go out to the market and come back with shopping, but strangers from the street cannot walk in.

Detailed answer

- Gives instances in **private subnets outbound-only** internet access (OS updates, calling external APIs, pulling public images).
- Inbound connections started from the internet are **not allowed** through it.
- It lives in a **public subnet**, has an **Elastic IP**, and private route tables point `0.0.0.0/0 -> nat-xxxx`.
- It is **zonal**: create **one per AZ** so an AZ failure does not cut off the other AZs (and to avoid cross-AZ data charges).
- **Cost:** hourly charge plus a per-GB data processing charge (about $0.045/GB in many regions). Use free S3/DynamoDB gateway endpoints to keep that traffic off the NAT.
- Managed and scales automatically, unlike a self-managed NAT instance.
- For IPv6, the equivalent is an **Egress-Only Internet Gateway**.

Example:

```
private-a route table:  10.0.0.0/16 -> local
                        0.0.0.0/0   -> nat-0aaa (in public-a)
private-b route table:  0.0.0.0/0   -> nat-0bbb (in public-b)

aws ec2 create-nat-gateway --subnet-id subnet-public-a --allocation-id eipalloc-0abc
```

> ***What to say to the interviewer:*** *A NAT Gateway lets resources in private subnets reach the internet outbound while blocking inbound connections from the internet. I place one in a public subnet per AZ with an Elastic IP, point private route tables to it, and keep S3 and DynamoDB traffic on gateway endpoints because NAT charges per GB processed.*

Q5. What is the difference between a Security Group and a NACL?

Also asked as: NACL vs Security Group.

> **In simple words:** A Security Group is a bouncer at each person's door who remembers who they let in. A NACL is the security guard at the gate of the whole building who checks everyone, both coming in and going out, and remembers nothing.

Detailed answer

Feature	Security Group	NACL
Level	Instance / ENI	Subnet
State	Stateful (return traffic auto-allowed)	Stateless (must allow return traffic)
Rules	Allow only	Allow and Deny
Evaluation	All rules together	In number order, first match wins
Default	Deny all inbound, allow all outbound	Default NACL allows all

A common trap: with NACLs you must allow **ephemeral ports 1024-65535** outbound, or response traffic is dropped.

Example:

```
SG app-sg     : inbound 8080 from alb-sg (reference SG, not IP)
NACL private  : rule 100 ALLOW tcp 8080 from 10.0.1.0/24
                rule 110 ALLOW tcp 1024-65535 outbound (return traffic)
                rule 90  DENY  all from 203.0.113.50/32 (block a bad IP)
```

> ***What to say to the interviewer:*** *Security Groups are stateful, allow-only firewalls attached to instances, while NACLs are stateless subnet-level rules that support explicit deny and are evaluated in order. I use Security Groups for normal access control and NACLs only for broad subnet rules like blocking a malicious IP range.*

Q6. What is an IAM role?

> **In simple words:** An IAM role is like a visitor badge. Nobody owns it permanently; whoever is allowed picks it up, gets temporary access, and the badge expires automatically.

Detailed answer

- A role has **no long-term password or access keys**.
- It has two policies: a **trust policy** (who can assume it) and a **permission policy** (what they can do).
- When assumed, STS issues **temporary credentials** (default 1 hour).
- Used by EC2 (instance profile), Lambda (execution role), EKS pods (IRSA / Pod Identity), CI/CD (OIDC), and other accounts (cross-account).

Example:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "ec2.amazonaws.com" },
    "Action": "sts:AssumeRole"
  }]
}
```

What to say to the interviewer: *An IAM role is an identity with permissions but no permanent credentials; a trusted entity like an EC2 instance, Lambda or another account assumes it and gets short-lived STS credentials. I always prefer roles over access keys because nothing is stored and the credentials rotate automatically.*

Q7. What are the different S3 storage classes and their use cases?

In simple words: Like storing things at home: the kitchen shelf (fast, expensive), the attic (cheaper, a bit slower), and a storage unit in another city (very cheap, takes hours to get things back).

Detailed answer

Class	Use case	Notes
Standard	Frequently accessed data	Default, millisecond access
Intelligent-Tiering	Unknown/changing access patterns	Auto-moves objects, small monitoring fee
Standard-IA	Infrequent access, needs fast retrieval	30-day min, retrieval fee
One Zone-IA	Re-creatable infrequent data	Single AZ, cheaper
Glacier Instant Retrieval	Archives read about once a quarter	ms access, 90-day min
Glacier Flexible Retrieval	Backups, minutes-to-hours restore	90-day min
Glacier Deep Archive	Compliance data kept 7-10 years	Cheapest, 12-48h restore, 180-day min
Express One Zone	Very low-latency, high request rate	Single AZ, directory buckets

Example:

```
App uploads (hot)        -> Standard
Logs older than 30 days  -> Standard-IA
Logs older than 90 days  -> Glacier Flexible Retrieval
Audit data (7 years)     -> Glacier Deep Archive
```

What to say to the interviewer: *S3 has classes from Standard for hot data, through Infrequent Access and Intelligent-Tiering, down to Glacier tiers for archives. I pick based on how often the data is read and how fast it must come back, and I use lifecycle rules to move data down the tiers automatically.*

Q8. What is an S3 lifecycle policy?

Also asked as: Configure a lifecycle rule to move an object to Glacier after 60 days and delete it after 90 days.

In simple words: An automatic cleaning schedule for a warehouse: after 30 days move boxes to the back room, after a year send them to off-site storage, after 3 years throw them out.

Detailed answer

Lifecycle rules act on objects (optionally filtered by prefix or tag):

- **Transition actions:** move objects to a cheaper class after N days.
- **Expiration actions:** delete objects after N days.
- **Noncurrent version rules:** clean old versions in versioned buckets.
- **Abort incomplete multipart uploads:** removes hidden half-uploaded parts that you still pay for.

Example:

```
{
  "Rules": [{
    "ID": "logs-tiering",
    "Filter": { "Prefix": "logs/" },
    "Status": "Enabled",
    "Transitions": [
      { "Days": 30, "StorageClass": "STANDARD_IA" },
      { "Days": 90, "StorageClass": "GLACIER" }
    ],
    "Expiration": { "Days": 1095 },
    "NoncurrentVersionExpiration": { "NoncurrentDays": 30 },
    "AbortIncompleteMultipartUpload": { "DaysAfterInitiation": 7 }
  }]
}
```

```
aws s3api put-bucket-lifecycle-configuration --bucket my-logs \
  --lifecycle-configuration file://lifecycle.json
```

Follow-up: "Move an object to Glacier after 60 days and delete it after 90 days."

```
{
  "Rules": [{
    "ID": "glacier-60-delete-90",
    "Filter": { "Prefix": "" },
    "Status": "Enabled",
    "Transitions": [{ "Days": 60, "StorageClass": "GLACIER" }],
    "Expiration": { "Days": 90 }
  }]
}
```

```
resource "aws_s3_bucket_lifecycle_configuration" "logs" {
  bucket = aws_s3_bucket.logs.id

  rule {
    id     = "glacier-60-delete-90"
    status = "Enabled"
    filter {}

    transition {
      days          = 60
      storage_class = "GLACIER"
    }

    expiration {
      days = 90
    }
  }
}
```

Cost gotcha (mention this in the interview):

- Glacier Flexible Retrieval (`GLACIER`) has a **90-day minimum storage duration**. An object moved on day 60 and deleted on day 90 has stayed only 30 days in Glacier, so you pay an **early-deletion fee** as if it stayed the full 90 days there. Moving it may cost more than leaving it in Standard.
- Better options: just **expire at 90 days** with no transition, or use Standard-IA (30-day minimum), or Glacier Instant Retrieval (still 90-day minimum, so same issue).
- Lifecycle transitions also have per-request costs, and by default objects **smaller than 128 KB are not transitioned** (it is not cost-effective), so millions of tiny log files may stay in Standard.

> ***What to say to the interviewer:*** *A lifecycle policy automatically transitions objects to cheaper storage classes or deletes them after a set age. For example, I move logs to Standard-IA after 30 days, Glacier after 90 and delete them after 3 years, plus abort incomplete multipart uploads so we do not pay for hidden junk. For a Glacier-at-60, delete-at-90 rule I would warn that Glacier's 90-day minimum means an early-deletion charge, so simply expiring at 90 days is often cheaper.*

Q9. What is CloudFront, and when would you use it?

> **In simple words:** Instead of every customer driving to your one warehouse, you put copies of popular items in small shops in every city. CloudFront is that network of shops (edge locations).

Detailed answer

- CloudFront is AWS's **CDN** with hundreds of edge locations worldwide.
- It caches content near users, cutting latency and load on the origin.
- Origins: S3, ALB, EC2, API Gateway, Lambda function URLs, or any HTTP server.
- Extra benefits: free TLS via ACM, AWS WAF and Shield integration, Origin Access Control to keep S3 private, CloudFront Functions / Lambda@Edge for logic at the edge.

Use it for static websites, images/videos, software downloads, and even dynamic APIs (it keeps optimized connections back to the origin).

Example:

```
User (Mumbai) -> Edge (Mumbai) --cache hit--> response in ~20 ms
                        |
                  cache miss -> Origin (S3 in us-east-1) -> cached for next user
```

What to say to the interviewer: *CloudFront is AWS's CDN that caches content at edge locations close to users. I use it in front of S3 or an ALB to reduce latency, offload the origin, terminate TLS and attach WAF, and I keep the S3 bucket private using Origin Access Control.*

Q10. What is Route 53, and how is it different from CoreDNS?

In simple words: Route 53 is the public phone book of the whole city. CoreDNS is the internal extension directory inside one office building (your Kubernetes cluster).

Detailed answer

	Route 53	CoreDNS
What	Managed AWS DNS service	DNS server running as pods in Kubernetes
Scope	Public internet and private VPC zones	Inside one cluster
Resolves	`app.example.com`, domains you own	`my-svc.my-ns.svc.cluster.local`
Extras	Domain registration, health checks, routing policies	Service discovery, plugins, forwards to upstream DNS

Route 53 routing policies: simple, weighted, latency, failover, geolocation, geoproximity, multivalue, IP-based.

Example:

```
Browser -> Route 53: api.example.com -> ALB DNS name
Pod     -> CoreDNS : orders.prod.svc.cluster.local -> 172.20.14.9 (ClusterIP)
Pod     -> CoreDNS : rds.amazonaws.com -> forwards to VPC resolver (.2)
```

What to say to the interviewer: *Route 53 is AWS's managed DNS for public and private domains, with health checks and routing policies like failover and latency. CoreDNS runs inside Kubernetes and resolves service names to ClusterIPs for pod-to-pod discovery, forwarding external names to the VPC resolver.*

Q11. What is a read replica in RDS?

In simple words: The main database is the teacher writing on the board. Read replicas are students copying the board so other people can read from them without disturbing the teacher.

Detailed answer

- A read-only copy of the primary database, updated with **asynchronous replication**.
- Offloads read-heavy traffic (reports, dashboards, search).
- Can be in the same AZ, another AZ, or **another region** (cross-region DR).
- Because replication is async, replicas can lag (check `ReplicaLag` metric).
- A replica can be **promoted** to a standalone primary.

- Different from Multi-AZ standby: a classic Multi-AZ standby is synchronous and not readable.

Example:

```
aws rds create-db-instance-read-replica \
  --db-instance-identifier orders-replica-1 \
  --source-db-instance-identifier orders-prod \
  --db-instance-class db.r6g.large
```

What to say to the interviewer: *A read replica is an asynchronously replicated, read-only copy of an RDS instance used to scale reads and for cross-region DR. It differs from Multi-AZ, which is a synchronous standby for high availability, and a replica can be promoted to a primary if needed.*

Q12. Explain the architecture of Amazon ECS.

In simple words: ECS is a restaurant manager. The task definition is the recipe, a task is one cooked dish, a service makes sure there are always N dishes on the counter, and the cluster is the kitchen.

Detailed answer

- **Cluster:** logical group of capacity.
- **Task definition:** JSON blueprint (image, CPU/memory, ports, env, IAM task role, log config).
- **Task:** a running instance of a task definition (one or more containers).
- **Service:** keeps the desired number of tasks running, replaces failed ones, integrates with ALB/NLB, does rolling or blue/green deploys.
- **Launch types / capacity:** Fargate (serverless, no servers) or EC2 (your ASG with the ECS agent), managed through **capacity providers**.
- **Networking:** `awsvpc` mode gives each task its own ENI and Security Group.
- Control plane is fully managed by AWS; Service Connect / Cloud Map handle service discovery.

Example:

```
ECS Cluster "prod"
 +-- Service "orders" (desired=4) --> ALB target group
 |     +-- Task (Fargate, ENI 10.0.11.23) -> container orders:1.4.2
 |     +-- Task (Fargate, ENI 10.0.12.41)
 +-- Capacity providers: FARGATE, FARGATE_SPOT
```

What to say to the interviewer: *ECS has clusters, task definitions that describe containers, tasks that are running copies, and services that keep a desired count running behind a load balancer. Capacity comes from Fargate or EC2 instances via capacity providers, and AWS fully manages the control plane.*

Q13. Why use Amazon EKS instead of ECS? What are the key differences?

Also asked as: ECS vs EKS.

In simple words: ECS is an automatic car from one brand: simple, just drive. EKS is a manual sports car that works the same everywhere: more control and portability, but you must know how to drive it.

Detailed answer

	ECS	EKS
Orchestrator	AWS proprietary	Upstream Kubernetes
Learning curve	Low	High
Portability	AWS only	Any cloud / on-prem
Control plane cost	Free	About $0.10/hour per cluster
Ecosystem	AWS integrations	Helm, ArgoCD, operators, service mesh, KEDA
Ops effort	Low	Higher (upgrades, add-ons, CNI)

Choose **ECS** for a small team, AWS-only, simple microservices. Choose **EKS** when you need Kubernetes tooling, multi-cloud portability, custom operators, or already have Kubernetes skills.

Example:

```
Startup, 10 services, AWS only          -> ECS on Fargate
Platform team, 100+ services, GitOps,
Helm charts, Kafka operators, hybrid     -> EKS (+ Karpenter, ArgoCD)
```

> ***What to say to the interviewer:*** *ECS is simpler and AWS-native with no control plane fee, while EKS gives standard Kubernetes with its huge ecosystem and portability at the cost of more operational work. I choose EKS when we need Helm, GitOps, operators or multi-cloud, and ECS when simplicity matters more.*

Q14. Which services can be integrated with a CDN (CloudFront)?

> **In simple words:** A CDN can sit in front of almost anything that speaks HTTP, and it can call helpers for security and small bits of logic.

Detailed answer

Origins CloudFront can serve from:

- Amazon S3 (with Origin Access Control)
- Application Load Balancer, EC2, NLB (VPC origins allow private ALB/EC2)
- API Gateway and Lambda function URLs
- AWS Elemental MediaPackage / MediaStore (video streaming)
- Any custom HTTP origin, even on-premises

Services that plug into CloudFront:

- AWS WAF (filter attacks), AWS Shield (DDoS)
- ACM (free TLS certificates, must be in us-east-1)
- CloudFront Functions and Lambda@Edge (redirects, headers, auth)
- Route 53 (alias records), CloudWatch and S3 for logs

Example:

```
Route53 (alias) -> CloudFront + WAF + ACM cert
                     |-- /static/*  -> S3 bucket (OAC)
                     |-- /api/*     -> ALB -> EKS
                     +-- /img/*     -> Lambda@Edge resize -> S3
```

What to say to the interviewer: *CloudFront integrates with S3, ALB, EC2, API Gateway, Lambda function URLs, media services and any custom HTTP origin. Around it I add WAF, Shield, ACM certificates and edge functions, with Route 53 alias records pointing the domain to the distribution.*

Q15. What is the default port for DynamoDB?

In simple words: DynamoDB is not a database server you dial into on a special number like MySQL (3306). It is a web service: you talk to it like a website, over HTTPS.

Detailed answer

- DynamoDB is a **managed HTTPS API**. Clients (SDK/CLI) call endpoints like `dynamodb.ap-south-1.amazonaws.com` on **port 443**.
- There is **no database port** to open and no connection to keep, unlike RDS (MySQL 3306, PostgreSQL 5432).
- Every request is signed with **IAM credentials (SigV4)**; access is controlled by IAM policies, not passwords.
- **DynamoDB Local** (the downloadable version for testing) listens on **port 8000** by default.
- To reach it privately from private subnets, use a **gateway VPC endpoint** for DynamoDB (free), so no NAT is needed.

Example:

```
aws dynamodb list-tables --region ap-south-1          # HTTPS 443 to AWS endpoint

docker run -p 8000:8000 amazon/dynamodb-local
aws dynamodb list-tables --endpoint-url http://localhost:8000   # local testing
```

What to say to the interviewer: *DynamoDB has no traditional database port; it is an HTTPS API reached on port 443 with IAM-signed requests. DynamoDB Local uses port 8000 for testing, and in a VPC I reach DynamoDB privately through a free gateway endpoint.*

Q16. How do you add storage to an existing EC2 instance?

In simple words: Either plug in an extra hard disk (new EBS volume) or make the existing disk bigger (resize). Both can be done while the computer keeps running.

Detailed answer

Option A - Add a new EBS volume:

1. Create an EBS volume in the **same AZ** as the instance (gp3 recommended).
2. **Attach** it to the instance.
3. On the instance: `lsblk` to find the device (for example `/dev/nvme1n1`).
4. Format it (`mkfs`) - only for a new, empty volume.

5. Mount it and add it to `/etc/fstab` using the **UUID** and the `nofail` option, so a missing disk does not block boot.

Option B - Grow an existing volume (no downtime):

1. `aws ec2 modify-volume` to increase the size.
2. `growpart` to extend the partition.
3. `resize2fs` (ext4) or `xfs_growfs` (XFS) to extend the filesystem.

Take a **snapshot first**. You can grow a volume but not shrink it, and you must wait about 6 hours between modifications of the same volume.

Example:

```
aws ec2 create-volume --availability-zone ap-south-1a --size 100 --volume-type gp3
aws ec2 attach-volume --volume-id vol-0new --instance-id i-0abc --device /dev/sdf
lsblk
sudo mkfs -t xfs /dev/nvme1n1
sudo mkdir /data && sudo mount /dev/nvme1n1 /data
echo "UUID=$(sudo blkid -s UUID -o value /dev/nvme1n1) /data xfs defaults,nofail 0 2" \
  | sudo tee -a /etc/fstab

# Grow existing root volume
aws ec2 modify-volume --volume-id vol-0root --size 50
sudo growpart /dev/nvme0n1 1
sudo xfs_growfs -d /          # or: sudo resize2fs /dev/nvme0n1p1
```

> ***What to say to the interviewer:*** *I either create a new EBS volume in the same AZ, attach it, format, mount and add it to fstab by UUID with nofail, or I grow the existing volume with modify-volume and then growpart and resize2fs or xfs_growfs. Both work without downtime, and I snapshot before changing anything.*

Q17. What is the difference between an ALB and an NLB, and when would you prefer an NLB, especially with databases?

> **In simple words:** An ALB is a smart receptionist who reads each letter and sends it to the right department. An NLB is a very fast mail chute that just forwards envelopes without opening them, at huge volume, from a fixed address.

Detailed answer

	ALB	NLB
OSI layer	Layer 7 (HTTP/HTTPS, gRPC, WebSocket)	Layer 4 (TCP, UDP, TLS)
Routing	Host, path, header, query, method rules	Port-based only
Static IP	No (DNS name only)	Yes, one static IP per AZ, can use Elastic IPs
Performance	Good, adds a few ms	Ultra-low latency, millions of requests/sec
Client IP	In `X-Forwarded-For` header	Preserved at the packet level
Extras	WAF, Cognito/OIDC auth, redirects, fixed responses	PrivateLink endpoint services, TLS passthrough
Targets	Instances, IPs, Lambda	Instances, IPs, ALB

Prefer an NLB when:

- The protocol is not HTTP (databases, MQTT, gaming, UDP, SMTP).
- A partner must **allowlist fixed IPs**.
- You need **PrivateLink**: an endpoint service must sit behind an NLB (or GWLB).
- You need extreme throughput or the original client IP.

Databases:

- Fronting a **self-managed DB cluster** on EC2 (for example PostgreSQL/Kafka/Redis nodes) with a TCP NLB.
- Exposing **RDS privately to another VPC or account via PrivateLink**: NLB with **IP targets** pointing to the RDS IP (keep it updated on failover, since the IP changes) or use RDS Proxy.
- For normal use **RDS needs no load balancer**: the endpoint DNS name plus Multi-AZ failover handles it; reads go to the reader endpoint.

GWLB (Gateway Load Balancer): Layer 3, used to insert firewalls/IDS appliances transparently (GENEVE), for example in an inspection VPC.

Example:

```
aws elbv2 create-load-balancer --name db-nlb --type network --scheme internal \
  --subnets subnet-a subnet-b
aws elbv2 create-target-group --name pg-tg --protocol TCP --port 5432 \
  --vpc-id vpc-0abc --target-type ip
aws ec2 create-vpc-endpoint-service-configuration \
  --network-load-balancer-arns <nlb-arn> --acceptance-required
```

> ***What to say to the interviewer:*** *The ALB is a layer 7 HTTP load balancer with path and host routing, WAF and auth, while the NLB is layer 4 for TCP, UDP and TLS with static IPs, very low latency and client IP preservation. I choose an NLB for non-HTTP protocols like databases, for fixed-IP allowlisting and for PrivateLink, for example to share a database with another account, though plain RDS does not need a load balancer because its endpoint and Multi-AZ failover handle routing.*

Q18. What is AWS KMS used for, and what should be stored in it?

In simple words: KMS is a bank that keeps the master keys to your safes; it never holds your valuables (passwords, data) itself. Your secrets live in a safe (Secrets Manager), and KMS holds the key that locks that safe.

Detailed answer

Common misconception: KMS stores and manages **encryption keys**, not secrets or passwords. Store secrets in **Secrets Manager** or **Parameter Store SecureString**, which KMS encrypts.

- **Key types: AWS managed keys** (`aws/s3`, `aws/ebs`, created and rotated by AWS, limited control) vs **customer managed keys (CMKs)** (you control key policy, rotation, deletion, cross-account use).
- **Key policies and grants** decide who can use or manage a key; a key policy is required and is the primary access control.
- **Rotation:** automatic yearly rotation (period configurable) for symmetric CMKs; old versions still decrypt old data.
- **Envelope encryption:** KMS generates a **data key**; the service encrypts your data locally with the data key and stores the data key encrypted by the KMS key. That is how large data is encrypted.
- **Direct Encrypt API** handles only up to **4 KB** of data, so for larger data use `GenerateDataKey`.
- **Integrations:** S3 SSE-KMS, EBS, RDS, EFS, DynamoDB, Secrets Manager, CloudWatch Logs, EKS secrets encryption.
- **Audit:** every use of the key is logged in **CloudTrail**; key deletion has a 7-30 day waiting period.

Example:

```
aws kms create-key --description "prod app data" --key-usage ENCRYPT_DECRYPT
aws kms create-alias --alias-name alias/prod-app --target-key-id <key-id>
aws kms enable-key-rotation --key-id <key-id>

aws s3api put-bucket-encryption --bucket app-data --server-side-encryption-configuration \
  '{"Rules":[{"ApplyServerSideEncryptionByDefault":
    {"SSEAlgorithm":"aws:kms","KMSMasterKeyID":"alias/prod-app"},"BucketKeyEnabled":true}]}'

aws kms generate-data-key --key-id alias/prod-app --key-spec AES_256   # envelope encryption
```

What to say to the interviewer: *KMS creates and manages encryption keys, not secrets; secrets belong in Secrets Manager or Parameter Store, which are encrypted by KMS. I use customer managed keys with tight key policies and rotation, services like S3, EBS and RDS use envelope encryption with data keys because direct Encrypt is limited to 4 KB, and CloudTrail audits every key use.*

Advanced

Q19. How do you allow one AWS account to safely access resources in another account (cross-account access)?

Also asked as: How do you secure cross-account access using IAM roles?

> **In simple words:** Account B (the house owner) makes a special key that only Account A is allowed to borrow. Account A borrows it, uses it for one hour to do only the allowed job, and the key disappears.

Detailed answer

Steps using **cross-account IAM roles**:

1. **Account B (target)** creates a role, for example `ProdS3ReadRole`.
2. Its **trust policy** names Account A (ideally a specific role, not the whole account) and can require an `ExternalId` or MFA.
3. Its **permission policy** gives least privilege, for example only `s3:GetObject` on one bucket.
4. **Account A** gives its user/role permission to call `sts:AssumeRole` on that role ARN.
5. Account A calls STS and receives temporary credentials (15 min to 12 h).

Why it is secure: no stored keys, credentials expire, least privilege, revocable instantly by editing the trust policy, and every call is logged in **CloudTrail** in both accounts.

Example:

```
{
  "Effect": "Allow",
  "Principal": { "AWS": "arn:aws:iam::111111111111:role/jenkins-ci" },
  "Action": "sts:AssumeRole",
  "Condition": { "StringEquals": { "sts:ExternalId": "ci-prod-2026" } }
}
```

```
aws sts assume-role \
  --role-arn arn:aws:iam::222222222222:role/ProdS3ReadRole \
  --role-session-name ci-cd --external-id ci-prod-2026
```

> ***What to say to the interviewer:*** *I create a role in the target account whose trust policy allows only a specific role in the source account, with least-privilege permissions. The source assumes it through STS to get temporary credentials, so nothing is stored, access expires automatically, and CloudTrail audits every call.*

Q20. How would you access data in an S3 bucket in Account A from an application running on EC2 in Account B? What permissions are needed on the bucket side?

Also asked as: An app in one AWS account needs to access an S3 bucket in another account - how do you set it up securely? How do you provide access to an S3 bucket?

> **In simple words:** For cross-account access, both sides must say yes: the visitor's company must allow the trip (IAM policy in B), and the building owner must allow the visitor in (bucket policy in A).

Detailed answer

Option 1 - Bucket policy + instance role (simple, direct):

1. EC2 in Account B has an **instance profile role** `app-role`.
2. `app-role` identity policy allows `s3:GetObject`/`s3:ListBucket` on the bucket ARN.
3. **Bucket policy in Account A** allows principal `arn:aws:iam::B:role/app-role`.
4. If the bucket uses **SSE-KMS**, the KMS key policy in A must also allow the role to `kms:Decrypt`.
5. For uploads, set Object Ownership to **BucketOwnerEnforced** so Account A owns objects.

Option 2 - Assume a role in Account A: Account A creates a role with S3 access trusting B's role; the app calls `sts:AssumeRole`. Better when access covers many resources in A.

Option 3 - Pre-signed URLs: someone with access in Account A generates a time-limited URL for one object. Good for one-off downloads/uploads by users or systems that have no AWS identity.

Option	Use when
Bucket policy + cross-account principal	One bucket, steady app access, simplest to audit
IAM role + AssumeRole	Many resources in Account A, or Account A wants full control of permissions
Pre-signed URL	Temporary access to a single object, no AWS identity on the client

Never copy access keys onto the instance.

Example (bucket policy in Account A):

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "AWS": "arn:aws:iam::222222222222:role/app-role" },
    "Action": ["s3:GetObject", "s3:ListBucket"],
    "Resource": ["arn:aws:s3:::reports-bucket", "arn:aws:s3:::reports-bucket/*"]
  }]
}
```

```
# Option 3: pre-signed URL valid for 1 hour
aws s3 presign s3://reports-bucket/2026/09/report.pdf --expires-in 3600
```

> ***What to say to the interviewer:*** *I give the EC2 instance an IAM role whose policy allows the needed S3 actions, and in Account A I add a bucket policy trusting that exact role ARN, because cross-account needs both sides to allow it. If the bucket is KMS-encrypted I also update the key policy, and I never place access keys on the server.*

Q21. How do you secure S3 buckets in a multi-account setup?

> **In simple words:** Lock every door by default, give named keys only to people who need them, and install cameras everywhere.

Detailed answer

- **Block Public Access** at the **account level** in every account (and enforce with an SCP so no one can turn it off).
- **SCPs** in AWS Organizations: deny `s3:PutBucketPolicy` that makes buckets public, deny unencrypted uploads.
- **Bucket policies** using `aws:PrincipalOrgID` so only principals from your organization can access.
- **Encryption:** SSE-KMS with customer-managed keys; key policies per account.
- **Deny non-TLS** requests with `aws:SecureTransport`.
- **Object Ownership = BucketOwnerEnforced** (disables ACLs).
- **VPC endpoint policies** and `aws:SourceVpce` conditions for private-only buckets.
- **Visibility:** CloudTrail data events, S3 server access logs, IAM Access Analyzer, AWS Config rules, Macie for sensitive data.
- Central **log-archive account** with Object Lock for audit logs.

Example:

```
{
  "Effect": "Deny",
  "Principal": "*",
  "Action": "s3:*",
  "Resource": ["arn:aws:s3:::shared-data", "arn:aws:s3:::shared-data/*"],
  "Condition": {
    "StringNotEquals": { "aws:PrincipalOrgID": "o-abc123xyz" }
  }
}
```

> ***What to say to the interviewer:*** *I enforce Block Public Access account-wide through SCPs, restrict bucket policies to our organization with aws:PrincipalOrgID, encrypt with KMS and deny non-TLS requests. Then I monitor with CloudTrail, Access Analyzer, Config and Macie so any drift is caught quickly.*

Q22. How can you access an EC2 instance that does not have a public IP?

> **In simple words:** The house has no front door on the street, but you can still get in through a secure intercom (SSM), a guarded side gate (EC2 Instance Connect Endpoint), or a guard house (bastion).

Detailed answer

1. **SSM Session Manager (best):** needs the SSM agent, an instance role with `AmazonSSMManagedInstanceCore`, and a path to SSM (NAT or VPC interface endpoints `ssm`, `ssmmessages`, `ec2messages`). No port 22, no keys, sessions logged.
2. **EC2 Instance Connect Endpoint:** a VPC endpoint that tunnels SSH/RDP to private IPs; access controlled by IAM.
3. **Bastion host** in a public subnet (older approach; must be hardened).
4. **VPN / Direct Connect** from the office network.

Example:

```
# Session Manager
aws ssm start-session --target i-0abc123def4567890

# EC2 Instance Connect Endpoint
aws ec2-instance-connect ssh --instance-id i-0abc123def4567890 \
  --connection-type eice
```

What to say to the interviewer: *My first choice is SSM Session Manager, which needs only the agent, an instance role and SSM endpoints, with no open SSH port and full session logging. Alternatives are EC2 Instance Connect Endpoint, a VPN, or a hardened bastion host as a last resort.*

Q23. An EC2 instance in a private subnet must download packages without a NAT Gateway or bastion host. How?

Also asked as: You have an EC2 in a private subnet and cannot use a NAT Gateway - how else can it reach the internet for updates? Which AWS services can help?

In simple words: If you cannot go out to the market, have the market deliver to your door through private delivery tunnels (VPC endpoints), or keep a pantry stocked in advance (baked AMI / internal mirror).

Detailed answer

- **S3 Gateway Endpoint (free):** Amazon Linux 2/2023 repositories are hosted on S3, so `yum`/`dnf` works through a gateway endpoint with no internet.
- **Interface endpoints (PrivateLink):** ECR (`ecr.api`, `ecr.dkr`) for images, SSM for patching via **Patch Manager**, CodeArtifact for pip/npm/Maven packages proxied from public repos.
- **Internal mirror:** sync packages into your own S3 bucket or Artifactory/Nexus and point the package manager there.
- **Bake an AMI** with EC2 Image Builder or Packer so packages are already installed.
- **Proxy instance** (Squid) in a public subnet with a domain allowlist.
- **Verification:** package managers check GPG signatures; also verify checksums of downloaded files.

Example:

```
aws ec2 create-vpc-endpoint --vpc-id vpc-0abc \
  --service-name com.amazonaws.ap-south-1.s3 \
  --vpc-endpoint-type Gateway --route-table-ids rtb-0private

# then on the instance (Amazon Linux 2023)
sudo dnf update -y
```

What to say to the interviewer: *For Amazon Linux I add a free S3 gateway endpoint because the OS repos live in S3, and I use interface endpoints for ECR, SSM Patch Manager and CodeArtifact for language packages. For other distros I use an internal mirror or bake packages into the AMI, and GPG signatures verify integrity.*

Q24. What is VPC peering and how does it work? If VPC A is peered with B, and B with C, can A talk to C?

Also asked as: How do you connect two VPCs?

> **In simple words:** You are friends with Bob, and Bob is friends with Carol. That does not make you friends with Carol. VPC peering is the same: a private road between exactly two VPCs, not transitive.

Detailed answer

How peering works:

1. VPC A sends a **peering request** to VPC B (same or different account/region); B **accepts**.
2. Add **routes on both sides**: A's route tables send B's CIDR to `pcx-xxxx`, and B's route tables send A's CIDR back.
3. Update **Security Groups** to allow the other VPC's CIDR (or referenced SGs in the same region).
4. Optionally enable **DNS resolution** across the peering so private hostnames resolve.
5. Traffic stays on the AWS backbone, no gateway or bandwidth bottleneck, and CIDRs must not overlap.

Transitivity:

- **No, A cannot reach C.** VPC peering is **non-transitive**. Traffic cannot hop through B.
- Also no edge-to-edge routing: A cannot use B's NAT Gateway, IGW, VPN or Direct Connect.

Solutions for A <-> C:

1. Create a direct **A <-> C peering** (fine for a few VPCs).
2. Use **AWS Transit Gateway** as a hub (scales to thousands of VPCs, supports route tables and segmentation).
3. **PrivateLink** if you only need to expose one service, not whole networks.

Example:

```
aws ec2 create-vpc-peering-connection --vpc-id vpc-A --peer-vpc-id vpc-B
aws ec2 accept-vpc-peering-connection --vpc-peering-connection-id pcx-0abc
aws ec2 create-route --route-table-id rtb-A --destination-cidr-block 10.20.0.0/16 \
  --vpc-peering-connection-id pcx-0abc
aws ec2 create-route --route-table-id rtb-B --destination-cidr-block 10.10.0.0/16 \
  --vpc-peering-connection-id pcx-0abc

Peering (non-transitive):    A <---> B <---> C      A -X-> C

Transit Gateway (hub):             TGW
                                 /  |  \
                                A   B   C     A -> TGW -> C  works
```

> ***What to say to the interviewer:*** *VPC peering is a private one-to-one connection: one side requests, the other accepts, and both sides add routes and security group rules for each other's non-overlapping CIDR. It is not transitive, so A cannot reach C through B; I would either add a direct A-C peering or, at scale, use a Transit Gateway as a central hub with route tables, and PrivateLink if only one service needs exposing.*

Q25. How can Instance 2 (with a static IP) communicate with Instance 1, which is in a private subnet behind a multi-AZ load balancer?

In simple words: Instance 1 lives in a back room. Instance 2 never visits it directly; it rings the reception desk (the load balancer), and reception passes the message inside.

Detailed answer

- Instance 2 always talks to the **load balancer**, never directly to Instance 1's private IP.
- If Instance 2 is outside AWS or in another VPC over the internet: use an **internet-facing ALB/NLB** in public subnets across AZs. Allow Instance 2's **Elastic IP /32** in the LB security group.
- The target (Instance 1) security group allows traffic **only from the LB security group**.
- If the partner needs a fixed IP to allowlist on their side, use an **NLB with Elastic IPs per AZ** (or Global Accelerator).
- If Instance 2 is in the same or a peered VPC / TGW, use an **internal** load balancer and route privately.
- Instance 1 replies through the LB, so it needs no public IP or NAT for this flow.

Example:

```
Instance2 (EIP 3.110.10.5)
     |  HTTPS 443
     v
Internet-facing ALB (public-a, public-b)   SG: allow 443 from 3.110.10.5/32
     |  8080
     v
Instance1 (private-a 10.0.11.20)           SG: allow 8080 from alb-sg only
```

What to say to the interviewer: *Instance 2 calls the load balancer's DNS name, and the load balancer forwards to Instance 1 in the private subnet. I allow only Instance 2's static IP on the load balancer security group and only the load balancer security group on Instance 1, using an internal load balancer instead if both are in connected VPCs.*

Q26. Your application is hosted in S3 and users are in different countries. How do you reduce latency?

In simple words: Do not make everyone fly to one city to pick up a parcel; open pickup points in every country.

Detailed answer

1. Put **CloudFront** in front of the S3 bucket; objects are cached at edge locations near users.
2. Keep the bucket private with **Origin Access Control (OAC)**.
3. Set good **Cache-Control** headers (long TTL for versioned assets like `app.3f2a.js`).
4. Enable **compression** (gzip/Brotli) and HTTP/2 or HTTP/3.
5. For uploads from far away, use **S3 Transfer Acceleration** or CloudFront PUT.
6. For truly regional data, use **S3 Cross-Region Replication** plus latency-based routing.
7. Invalidate the cache on deploy (`/index.html`) instead of short TTLs everywhere.

Example:

```
aws cloudfront create-invalidation --distribution-id E1ABC2DEF3 --paths "/index.html"

Before: user in Sydney -> S3 us-east-1        ~250 ms
After : user in Sydney -> CloudFront Sydney   ~15 ms (cache hit)
```

> ***What to say to the interviewer:*** *I put CloudFront in front of the S3 bucket with Origin Access Control, so content is cached close to users worldwide. I tune Cache-Control headers, enable compression, invalidate on deploy, and for uploads use Transfer Acceleration or cross-region replication.*

Q27. You want to serve users in different countries. How do you route traffic based on user location?

> **In simple words:** Like a call center that routes your call to the office in your own country or the one with the shortest queue.

Detailed answer

Route 53 routing policies:

- **Geolocation:** route by user country/continent (for example, EU users -> eu-west-1 for GDPR). Always add a **default** record.
- **Latency-based:** send users to the region with the lowest measured latency.
- **Geoproximity:** by distance, with a bias to shift more or less traffic to a region.
- Attach **health checks** so unhealthy regions are skipped automatically.

Other options: **CloudFront** (edge caching, geo restriction), **Global Accelerator** (anycast static IPs, routes to nearest healthy region over the AWS backbone).

Example:

```
api.example.com
  Geolocation: Europe        -> ALB eu-west-1     (health check)
  Geolocation: Asia          -> ALB ap-south-1    (health check)
  Geolocation: Default       -> ALB us-east-1
```

> ***What to say to the interviewer:*** *I use Route 53 geolocation routing when data must stay in a region, or latency-based routing for best performance, always with health checks and a default record. For non-DNS routing I use Global Accelerator, and CloudFront for cached content.*

Q28. If the primary RDS database fails, how can you promote a read replica?

> **In simple words:** The team captain is injured, so you give the armband to the vice-captain. He becomes the new captain, but he may not know the very last play (async lag).

Detailed answer

1. Check **ReplicaLag**; replication is async, so recent writes may be lost.
2. Stop writes to the old primary if it is partially alive (avoid split brain).
3. Promote: the replica reboots and becomes a **standalone, writable** instance (no longer replicating).

4. Enable **Multi-AZ** and automated backups on the new primary.
5. **Repoint applications:** update a Route 53 CNAME (for example `db.internal`), Secrets Manager endpoint, or config.
6. Create new read replicas from the new primary.
7. For **Aurora**, failover to a replica is automatic and keeps the same cluster endpoint.

Example:

```
aws rds describe-db-instances --db-instance-identifier orders-replica-1 \
  --query "DBInstances[0].StatusInfos"
aws rds promote-read-replica --db-instance-identifier orders-replica-1 \
  --backup-retention-period 7
aws rds modify-db-instance --db-instance-identifier orders-replica-1 \
  --multi-az --apply-immediately
```

> ***What to say to the interviewer:*** *I check replica lag, fence the old primary, and run promote-read-replica, which makes the replica a standalone writable database. Then I repoint apps via a DNS CNAME or secret, enable Multi-AZ and backups, and rebuild replicas; with Aurora this failover is automatic.*

Q29. How do you handle database failover in RDS Multi-AZ?

> **In simple words:** A standby goalkeeper stands next to the goal, copying every move in real time. If the main keeper falls, the standby takes over within a minute, and the scoreboard (DNS name) stays the same.

Detailed answer

- **Multi-AZ instance:** a **synchronous** standby in another AZ; no data loss on failover.
- **Automatic failover** triggers on AZ outage, primary host failure, storage failure, or reboot-with-failover. Typical time **60-120 seconds**.
- The **endpoint DNS name does not change**; RDS flips the CNAME to the standby.
- **Multi-AZ DB cluster** (2 readable standbys) fails over faster, typically under 35 seconds.

What you must do on the app side:

- Use the endpoint name, never an IP; keep **DNS TTL low** (JVM: set `networkaddress.cache.ttl`).
- **Retry logic** with backoff and connection pool validation.
- Use **RDS Proxy** to shorten failover impact and keep connections.
- Subscribe to **RDS event notifications** (SNS) and **test failover** regularly.

Example:

```
aws rds reboot-db-instance --db-instance-identifier orders-prod --force-failover

aws rds create-event-subscription --subscription-name rds-failover \
  --sns-topic-arn arn:aws:sns:ap-south-1:111111111111:db-alerts \
  --source-type db-instance --event-categories failover
```

> ***What to say to the interviewer:*** *RDS Multi-AZ keeps a synchronous standby in another AZ and fails over automatically in about one to two minutes by flipping the endpoint DNS. I make apps*

resilient with retries, low DNS TTL and RDS Proxy, subscribe to failover events, and test with reboot --force-failover.

Q30. How can a developer securely access a private RDS instance without a bastion host?

In simple words: Instead of building a guard house, give the developer a temporary, recorded, private tunnel that only their identity can open.

Detailed answer

1. **SSM Session Manager port forwarding** through any SSM-managed instance or container in the VPC (no inbound ports, no SSH keys, IAM-controlled, logged).
2. **EC2 Instance Connect Endpoint** can open a TCP tunnel to private IPs.
3. **AWS Client VPN** for teams that need network access.
4. Combine with **IAM database authentication** (15-minute tokens) or Secrets Manager credentials, so no shared static passwords.
5. Keep RDS SG allowing only the tunnel source; log sessions to CloudWatch/S3.

Example:

```
aws ssm start-session --target i-0abc123def4567890 \
  --document-name AWS-StartPortForwardingSessionToRemoteHost \
  --parameters '{"host":["orders.abc123.ap-south-1.rds.amazonaws.com"],
                 "portNumber":["5432"],"localPortNumber":["15432"]}'

TOKEN=$(aws rds generate-db-auth-token --hostname orders.abc123... \
  --port 5432 --username dev_ro)
psql "host=localhost port=15432 user=dev_ro password=$TOKEN sslmode=require"
```

What to say to the interviewer: *I use SSM Session Manager port forwarding to the RDS endpoint, so there are no open ports or SSH keys and every session is logged. Combined with IAM database authentication tokens, developers get short-lived, auditable access tied to their own identity.*

Q31. How do you configure priority-based Auto Scaling?

In simple words: When hiring extra staff, you first call your favorite agency; only if they have nobody free do you call the second agency, then the third.

Detailed answer

"Priority" in AWS Auto Scaling usually means one of these:

- **Mixed Instances Policy, On-Demand allocation strategy** `prioritized`**:** ASG launches instance types in the order you list them in overrides.
- **Spot allocation** `capacity-optimized-prioritized`**:** prefers your order but still picks pools with spare capacity.
- **Base + percentage:** for example first 2 instances On-Demand, then 30 percent On-Demand, 70 percent Spot.
- **Multiple scaling policies:** if several fire at once, the ASG applies the one that gives the **largest capacity** (safest).

- **ECS capacity providers:** `base` and `weight` decide which provider (Fargate vs Fargate Spot) gets tasks first.

Example:

```
"MixedInstancesPolicy": {
  "LaunchTemplate": {
    "LaunchTemplateSpecification": { "LaunchTemplateName": "app-lt", "Version": "$Latest" },
    "Overrides": [
      { "InstanceType": "m7g.large" },
      { "InstanceType": "m6g.large" },
      { "InstanceType": "m5.large" }
    ]
  },
  "InstancesDistribution": {
    "OnDemandAllocationStrategy": "prioritized",
    "OnDemandBaseCapacity": 2,
    "OnDemandPercentageAboveBaseCapacity": 30,
    "SpotAllocationStrategy": "capacity-optimized-prioritized"
  }
}
```

> ***What to say to the interviewer:*** *I use a mixed instances policy where the On-Demand allocation strategy is prioritized, so the ASG tries instance types in my listed order, with Spot using capacity-optimized-prioritized. I also set an On-Demand base for critical capacity and remember that when multiple scaling policies fire, AWS picks the one giving the largest capacity.*

Q32. How do you automate ECS and EKS node scaling?

> **In simple words:** Two layers of hiring: first decide how many workers (pods/tasks) you need, then automatically rent enough desks (nodes) for them.

Detailed answer

ECS:

- **Service Auto Scaling** scales task count (target tracking on CPU, memory, or ALB requests per target).
- **Capacity providers with managed scaling** scale the EC2 ASG using the `CapacityProviderReservation` metric; managed termination protection avoids killing busy instances.
- Or use **Fargate** and skip nodes entirely.

EKS:

- **HPA/KEDA** scale pods.
- **Cluster Autoscaler** adds nodes to managed node groups when pods are Pending, removes underused nodes.
- **Karpenter** (recommended) launches right-sized instances directly in seconds, supports Spot and consolidation.
- **EKS Auto Mode** lets AWS manage node scaling for you.

Example:

```
# Karpenter NodePool
apiVersion: karpenter.sh/v1
kind: NodePool
metadata: { name: default }
spec:
  template:
    spec:
      requirements:
        - { key: karpenter.sh/capacity-type, operator: In, values: ["spot","on-demand"] }
        - { key: kubernetes.io/arch, operator: In, values: ["arm64","amd64"] }
      nodeClassRef: { group: karpenter.k8s.aws, kind: EC2NodeClass, name: default }
  limits: { cpu: "200" }
  disruption: { consolidationPolicy: WhenEmptyOrUnderutilized }
```

What to say to the interviewer: *For ECS I combine Service Auto Scaling for tasks with capacity providers using managed scaling for the EC2 fleet, or use Fargate. For EKS, HPA or KEDA scale pods and Karpenter or Cluster Autoscaler add and remove nodes based on pending pods, with Karpenter also consolidating to save cost.*

Q33. How would you perform a zero-downtime deployment in AWS?

In simple words: Change the tires of a moving bus one at a time, or bring a second bus alongside and move passengers over only when it runs fine.

Detailed answer

Building blocks:

- An **ALB** in front, with **health checks** and a **deregistration delay** (connection draining) so in-flight requests finish.
- At least **2 instances/tasks across AZs**.

Strategies:

1. **Rolling:** ASG **instance refresh** or ECS rolling update with `minimumHealthyPercent=100`, `maximumPercent=200`.
2. **Blue/Green:** CodeDeploy for ECS/EC2 creates a green fleet, shifts ALB traffic (all at once, linear, or canary), and rolls back on CloudWatch alarms.
3. **Canary for Lambda:** aliases with weighted traffic shifting.
4. **Deployment circuit breaker** in ECS for automatic rollback.
5. Database changes must be **backward-compatible** (expand-contract).

Example:

```
aws autoscaling start-instance-refresh --auto-scaling-group-name web-asg \
  --preferences '{"MinHealthyPercentage":90,"InstanceWarmup":120,
                  "AutoRollback":true}'

aws ecs update-service --cluster prod --service orders \
  --deployment-configuration "deploymentCircuitBreaker={enable=true,rollback=true},
  maximumPercent=200,minimumHealthyPercent=100"
```

What to say to the interviewer: *I keep at least two targets behind an ALB with health checks and connection draining, then use rolling updates or CodeDeploy blue/green that shifts traffic gradually and rolls back on CloudWatch alarms. Database changes are made backward-compatible so old and*

new versions can run together.

Q34. How do you implement API Gateway with Lambda for microservices?

In simple words: API Gateway is the reception desk that checks visitors and sends each one to the right department (Lambda function). Departments only open when someone arrives.

Detailed answer

Design:

- **One Lambda per bounded context** (orders, users, payments), or one per route for small functions.
- **HTTP API** (cheaper, faster) or **REST API** (usage plans, API keys, request validation, WAF).
- **Auth:** Cognito / JWT authorizer or Lambda authorizer; IAM auth for service-to-service.
- Each function has its **own IAM role** (least privilege) and env config from SSM/Secrets Manager.
- **Data:** DynamoDB per service; async work through SQS/EventBridge.
- **Stages** (dev/prod) and **Lambda aliases** for safe deploys.
- **Observability:** X-Ray tracing, structured logs, CloudWatch alarms on 5XX and latency.
- Deploy with SAM, CDK, or Terraform.

Example:

```
Client -> API Gateway (JWT authorizer, throttling, WAF)
            |-- GET  /orders/{id}  -> Lambda orders-read  -> DynamoDB orders
            |-- POST /orders       -> Lambda orders-write -> DynamoDB + EventBridge
            +-- POST /payments     -> Lambda payments     -> SQS -> Lambda worker
```

What to say to the interviewer: *I front microservices with API Gateway routes mapped to separate Lambda functions, each with its own least-privilege role and data store, using JWT or Lambda authorizers for auth. Async flows go through SQS or EventBridge, and I use stages, aliases and X-Ray for safe deploys and tracing.*

Q35. How do you handle rate limiting and throttling in API Gateway?

In simple words: A tap with a flow limiter. It lets a steady stream through (rate) and allows a small splash at once (burst). Extra water is politely refused with "try again later" (HTTP 429).

Detailed answer

- API Gateway uses a **token bucket**: **rate** (requests/second) and **burst** (max at once).
- **Account-level** default per region: 10,000 rps steady, 5,000 burst (soft limit).
- **Stage and method throttling:** protect expensive endpoints, for example `POST /reports` at 20 rps.
- **Usage plans + API keys** (REST API): per-customer quotas like 1,000 requests/day.
- **AWS WAF rate-based rules** block abusive IPs.
- Protect the backend too: **Lambda reserved concurrency**, SQS buffering, DynamoDB on-demand.

- Clients should retry with **exponential backoff and jitter** on 429.
- Monitor the `Count`, `4XXError`, and Lambda `Throttles` metrics.

Example:

```
aws apigateway update-stage --rest-api-id a1b2c3 --stage-name prod \
  --patch-operations \
  op=replace,path=/*/*/throttling/rateLimit,value=500 \
  op=replace,path=/*/*/throttling/burstLimit,value=1000

aws lambda put-function-concurrency --function-name reports \
  --reserved-concurrent-executions 50
```

> ***What to say to the interviewer:*** *API Gateway throttles with a token bucket at account, stage and method levels, and REST APIs add usage plans with per-key quotas that return 429 when exceeded. I also add WAF rate-based rules, reserved concurrency on Lambda, and make clients retry with exponential backoff.*

Q36. What are best practices for AWS serverless services (Lambda, API Gateway, DynamoDB)?

> **In simple words:** Serverless is like taking taxis instead of owning cars. Cheap and easy, but plan for traffic jams (throttles), short trips (timeouts), and not leaving the meter running.

Detailed answer

Lambda:

- Small, single-purpose functions; initialize SDK clients **outside the handler** to reuse connections.
- Right-size memory (also gives more CPU); use **Graviton (arm64)**.
- Set sensible timeouts, **reserved concurrency** to protect downstreams, **provisioned concurrency** or SnapStart for cold-start-sensitive APIs.
- Make handlers **idempotent**; use **DLQs / on-failure destinations**.
- Least-privilege role per function; secrets from Secrets Manager, not plain env vars.

API Gateway: throttling, request validation, authorizers, caching, WAF.

DynamoDB: design around access patterns and a good partition key (avoid hot keys), on-demand for spiky traffic, TTL for expiring data, PITR backups, GSIs carefully.

Ops: IaC (SAM/CDK/Terraform), X-Ray, structured logs, alarms on Errors, Throttles, Duration.

Example:

```
import boto3
ddb = boto3.resource("dynamodb").Table("orders")   # created once, reused

def handler(event, context):
    ddb.put_item(Item={"pk": event["id"], "status": "NEW"},
                 ConditionExpression="attribute_not_exists(pk)")   # idempotent
```

> ***What to say to the interviewer:*** *I keep Lambdas small and idempotent, reuse connections outside the handler, right-size memory, and use reserved or provisioned concurrency and DLQs. API Gateway gets throttling and authorizers, and DynamoDB tables are designed around access*

patterns with good partition keys, TTL and PITR.

Q37. How would you design separate environments (Dev, Test, Prod) in AWS?

In simple words: Give each environment its own house rather than separate rooms in one house, so a fire in the test kitchen cannot reach production.

Detailed answer

- **Separate AWS accounts per environment** under **AWS Organizations** (strongest isolation of blast radius, billing and limits).
- **OU structure:** Security (log-archive, audit), Infrastructure (network, shared services), Workloads (dev, test, prod).
- **AWS Control Tower** or Account Factory for guardrails and account vending.
- **SCPs:** for example deny leaving approved regions, deny disabling CloudTrail, block public S3.
- **IAM Identity Center** for SSO; developers get broad access in dev, read-only in prod.
- **Non-overlapping CIDRs**, connected via Transit Gateway if needed.
- **Same IaC code**, different variables per env; promote the same artifact dev -> test -> prod.
- Tag everything (`env`, `owner`, `cost-center`); budgets per account.

Example:

```
Root (Organizations, SCPs)
 +-- Security OU      : log-archive, audit
 +-- Infrastructure OU: network (TGW), shared-services (ECR, CI)
 +-- Workloads OU
      +-- dev   (10.10.0.0/16)  developers: PowerUser
      +-- test  (10.20.0.0/16)
      +-- prod  (10.30.0.0/16)  developers: ReadOnly, deploy only via CI role
```

What to say to the interviewer: *I use a separate AWS account per environment under Organizations with OUs, SCP guardrails and SSO through IAM Identity Center. The same Terraform code with per-environment variables builds each one, CIDRs do not overlap, and only the CI/CD role can deploy to prod.*

Q38. What are best practices for VPC design at enterprise scale?

In simple words: Plan a city before building it: reserve non-overlapping street numbers, build roads in every district, and put one central highway interchange instead of private bridges between every pair of towns.

Detailed answer

- **IP planning:** non-overlapping CIDRs across all accounts and on-prem; use **VPC IPAM**; leave room for growth (EKS pods consume many IPs).
- **3 AZs**, tiered subnets: public (LB, NAT), private app, isolated data.
- **One NAT Gateway per AZ** for resilience (or a centralized egress VPC to save cost).
- **Hub-and-spoke with Transit Gateway**; separate TGW route tables for prod and non-prod.
- **Shared VPCs via AWS RAM** so a network team owns networking and app teams deploy into it.

- **VPC endpoints** (S3/DynamoDB gateway, others via interface) to cut NAT cost and keep traffic private.
- **Security:** SG-to-SG references, NACLs for coarse rules, Network Firewall in an inspection VPC.
- **Visibility:** VPC Flow Logs, Route 53 Resolver query logs.
- **Hybrid:** Direct Connect with VPN backup; Route 53 Resolver endpoints for hybrid DNS.
- Everything in **IaC**.

Example:

```
         On-prem ==DX/VPN==+
                           |
Inspection VPC <---> Transit Gateway <---> Egress VPC (NAT x3)
                      /     |      \
                prod VPC  test VPC  shared-services VPC
                3 AZ x (public | app | data) subnets
```

> ***What to say to the interviewer:*** *I start with an IPAM-managed, non-overlapping CIDR plan, build VPCs across three AZs with public, app and data tiers, and connect them hub-and-spoke through Transit Gateway with separate route tables. I add VPC endpoints, centralized egress and inspection, flow logs, and manage it all with Terraform.*

Q39. How do you design a highly available architecture across Multi-AZ and Multi-Region?

Also asked as: Design a highly available backend on AWS.

> **In simple words:** Multi-AZ is having two kitchens in the same city so one fire does not stop dinner. Multi-Region is having a kitchen in another city in case the whole city loses power.

Detailed answer

Multi-AZ (default for production):

- ALB across 3 AZs; ASG/EKS nodes spread across AZs.
- RDS Multi-AZ, ElastiCache with replicas and auto-failover, MSK across 3 AZs.
- NAT Gateway per AZ; stateless apps; sessions in Redis/DynamoDB.

Multi-Region (for DR or global users):

- **Data replication:** Aurora Global Database, DynamoDB Global Tables, S3 Cross-Region Replication, cross-region RDS replicas.
- **Traffic:** Route 53 failover/latency records with health checks, or Global Accelerator.
- **Infra parity** through the same IaC in both regions; images replicated in ECR.
- Choose **active-passive** (cheaper) or **active-active** (complex, needs conflict handling).
- Regularly **test failover** (game days).

Example:

```
          Route 53 / Global Accelerator (health checks)
             /                                  \
ap-south-1 (primary)                     ap-southeast-1 (DR)
ALB -> EKS (3 AZ)                         ALB -> EKS (scaled down)
Aurora writer ======= Global DB =======> Aurora reader (promote on DR)
S3 bucket ============ CRR ============> S3 bucket
```

What to say to the interviewer: *Within a region I spread every layer across three AZs: ALB, autoscaled compute, Multi-AZ databases and per-AZ NAT, keeping apps stateless. For multi-region I replicate data with Aurora Global Database, DynamoDB Global Tables and S3 CRR, deploy the same IaC in both regions, and fail over with Route 53 health checks or Global Accelerator.*

Q40. What is your strategy for Disaster Recovery and Backup in AWS?

Also asked as: Do you have disaster recovery experience? What is your disaster recovery plan for a regional outage?

In simple words: Decide two numbers first: how much data you can afford to lose (RPO) and how long you can be down (RTO). Then pick the cheapest plan that meets them.

Detailed answer

- **RPO (Recovery Point Objective):** the maximum data loss you accept, measured in time (for example "the last 5 minutes of orders").
- **RTO (Recovery Time Objective):** the maximum time the service may be down before it is back.

Strategy	RPO / RTO	Cost	How
Backup and Restore	hours / hours	Lowest	Snapshots copied to another region, rebuild with IaC
Pilot Light	minutes / tens of minutes	Low	Data replicated live; core infra off or minimal
Warm Standby	seconds-minutes / minutes	Medium	Scaled-down full copy running
Active-Active	near zero / near zero	Highest	Both regions serve traffic

Backup practices:

- **AWS Backup** plans for EBS, RDS, DynamoDB, EFS, S3; cross-region and **cross-account copies**.
- **Vault Lock / Object Lock** to protect against ransomware or accidental deletion.
- RDS automated backups + PITR; Velero for Kubernetes objects and volumes.
- Everything rebuildable from **IaC** (see the Terraform chapter for multi-region modules, remote state and state backups).
- **Test restores regularly** and verify data (checksums, row counts). A backup never restored is only a hope.

Regional outage runbook (warm standby / pilot light):

1. **Declare the incident:** confirm via AWS Health Dashboard and your own health checks; the incident commander decides to fail over using pre-agreed criteria.

2. **Fail over data:** promote the **Aurora Global Database** secondary (or the RDS cross-region read replica); DynamoDB Global Tables are already active in both regions; S3 CRR buckets already hold the data.
3. **Bring up compute:** scale up the warm standby (ASG/EKS node pools) or apply the same IaC in the DR region for pilot light.
4. **Shift traffic:** Route 53 failover records with health checks, **Route 53 ARC routing controls** (data-plane switches built for this), or Global Accelerator traffic dials.
5. **Verify:** smoke tests, synthetic checks, dashboards, error rates.
6. **Communicate:** status page and stakeholder updates at fixed intervals.
7. **Fail back later** in a calm window after re-syncing data to the primary region.

Static stability: failover must not depend on the failed region's control plane (you should not need to edit resources in the broken region). Use data-plane mechanisms like ARC routing controls and pre-provisioned capacity. Run **game-day drills** regularly (for example quarterly) and record the real RTO/RPO achieved.

Example:

```
aws backup create-backup-plan --backup-plan '{
  "BackupPlanName": "prod-daily",
  "Rules": [{ "RuleName": "daily", "TargetBackupVaultName": "prod-vault",
    "ScheduleExpression": "cron(0 20 * * ? *)",
    "Lifecycle": { "DeleteAfterDays": 35 },
    "CopyActions": [{ "DestinationBackupVaultArn":
      "arn:aws:backup:ap-southeast-1:333333333333:backup-vault:dr-vault" }] }]}'
```

> ***What to say to the interviewer:*** *I start from the business RPO, how much data we can lose, and RTO, how long we can be down, and choose between backup-restore, pilot light, warm standby or active-active. I use AWS Backup with cross-region and cross-account copies and IaC to rebuild, and for a regional outage my runbook promotes the Aurora Global Database secondary, scales up the standby, shifts traffic with ARC routing controls or Route 53 failover, verifies and communicates, and we rehearse it in regular game days.*

Q41. How do you handle secret rotation using Secrets Manager or Parameter Store?

> **In simple words:** Change the locks on a schedule automatically, and make sure everyone always asks the key cabinet for the current key instead of keeping an old copy in their pocket.

Detailed answer

Secrets Manager (built for rotation):

- **Managed rotation** for RDS master passwords (`--manage-master-user-password`).
- **Rotation Lambda** for other secrets: steps `createSecret -> setSecret -> testSecret -> finishSecret`, using staging labels `AWSPENDING` and `AWSCURRENT`.
- **Alternating-users strategy** so connections using the old user keep working during rotation.
- Schedule, for example every 30 days.

Parameter Store: no native rotation; use EventBridge schedule + Lambda, or migrate rotating secrets to Secrets Manager.

Apps must read secrets at runtime (SDK with caching) or via External Secrets Operator / Secrets Store CSI on EKS, and reload on change.

Example:

```
aws secretsmanager rotate-secret --secret-id prod/orders/db \
  --rotation-lambda-arn arn:aws:lambda:ap-south-1:111111111111:function:pg-rotator \
  --rotation-rules '{"ScheduleExpression":"rate(30 days)"}'

aws rds modify-db-instance --db-instance-identifier orders-prod \
  --manage-master-user-password --apply-immediately
```

> ***What to say to the interviewer:*** *I use Secrets Manager rotation, managed for RDS and via a rotation Lambda with the alternating-users strategy for other databases, on a fixed schedule. Applications fetch secrets at runtime with caching, or through External Secrets Operator on EKS, so rotations happen without redeploys.*

Q42. What is your approach for centralized secrets management for 100+ microservices?

> **In simple words:** One secure bank vault with a separate locker per service. Each service's badge opens only its own locker, and the vault logs every opening.

Detailed answer

- **Single source of truth:** AWS Secrets Manager (or HashiCorp Vault) with a naming convention `/<env>/<service>/<name>`.
- **Identity-based access:** each service gets its own IAM role (IRSA or EKS Pod Identity, ECS task role) allowed only `secretsmanager:GetSecretValue` on its own path.
- **Delivery into apps:** External Secrets Operator syncs to Kubernetes Secrets, or Secrets Store CSI driver mounts them; ECS injects via `secrets` in the task definition.
- **Encryption** with a customer-managed KMS key per environment.
- **Rotation** automated; apps reload.
- **Cross-account** sharing via resource policies when needed.
- **Audit** with CloudTrail; alert on unusual access. Never store secrets in Git (use SOPS or sealed secrets for GitOps if needed).
- Enforce with policy (OPA/Kyverno: no plain env secrets).

Example:

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata: { name: orders-db, namespace: orders }
spec:
  refreshInterval: 1h
  secretStoreRef: { name: aws-secrets-manager, kind: ClusterSecretStore }
  target: { name: orders-db }
  data:
    - secretKey: password
      remoteRef: { key: prod/orders/db, property: password }
```

What to say to the interviewer: *I store all secrets centrally in Secrets Manager with a per-environment, per-service path and give each service its own IAM role scoped to its path through IRSA or task roles. External Secrets Operator delivers them into pods, KMS encrypts them, rotation is automated and CloudTrail audits access.*

Q43. What strategy do you use to rotate access keys automatically?

In simple words: The best key is no key at all. If you must have one, replace it on a timer and throw the old one away after everyone has the new one.

Detailed answer

1. **Eliminate long-lived keys first:** IAM roles for EC2/Lambda, IRSA/Pod Identity for EKS, **OIDC federation** for GitHub Actions/GitLab/Jenkins, IAM Identity Center for humans.
2. For unavoidable keys (legacy third-party tools):

- An **EventBridge scheduled Lambda** creates a new key (IAM allows 2 per user), stores it in **Secrets Manager**, lets consumers pick it up, then **deactivates** the old key after a grace period and **deletes** it later.
- Or use Secrets Manager rotation with a custom Lambda for IAM keys.

3. **Detect** with AWS Config rule `access-keys-rotated` (for example max 90 days) and IAM credential report; alert on keys unused for 90 days.
4. **SCP** can deny `iam:CreateAccessKey` except for approved roles.

Example:

```
aws iam create-access-key --user-name legacy-tool
aws secretsmanager put-secret-value --secret-id legacy-tool/keys --secret-string file://k.json
aws iam update-access-key --user-name legacy-tool --access-key-id AKIAOLD... --status Inactive
aws iam delete-access-key --user-name legacy-tool --access-key-id AKIAOLD...

aws configservice put-config-rule --config-rule '{"ConfigRuleName":"keys-rotated",
  "Source":{"Owner":"AWS","SourceIdentifier":"ACCESS_KEYS_ROTATED"},
  "InputParameters":"{\"maxAccessKeyAge\":\"90\"}"}'
```

What to say to the interviewer: *My main strategy is to remove access keys entirely using roles, IRSA and OIDC for CI/CD. For the few that must exist, a scheduled Lambda creates a new key, stores it in Secrets Manager, deactivates and later deletes the old one, and an AWS Config rule flags any key older than 90 days.*

Q44. What metrics do you monitor for EC2, ECS, and RDS using CloudWatch?

Also asked as: How do you monitor AWS resources with CloudWatch?

In simple words: Like a car dashboard: speed (CPU), fuel (memory/disk), engine warnings (status checks), and for the database, how many passengers (connections) and how slow the doors open (latency).

Detailed answer

Service	Key metrics
EC2	CPUUtilization, CPUCreditBalance (T types), StatusCheckFailed_System/Instance, NetworkIn/Out, EBS VolumeQueueLength, BurstBalance; memory and disk via CloudWatch Agent
ECS	Service CPUUtilization, MemoryUtilization, RunningTaskCount vs desired, Container Insights (restarts, task-level), ALB TargetResponseTime, HTTPCode_Target_5XX, UnHealthyHostCount
RDS	CPUUtilization, DatabaseConnections, FreeableMemory, FreeStorageSpace, Read/WriteLatency, Read/WriteIOPS, DiskQueueDepth, ReplicaLag, SwapUsage, BurstBalance; Performance Insights for top SQL

CloudWatch building blocks:

- **Metrics:** free default metrics per service (EC2 every 5 min, 1 min with detailed monitoring); custom metrics via `put-metric-data`.
- **CloudWatch Agent:** needed for **memory and disk** usage on EC2 (not sent by default) and to ship log files.
- **Alarms:** threshold or anomaly detection, composite alarms to reduce noise, actions to SNS, Auto Scaling, or EC2 recover.
- **Logs:** log groups with retention settings; **metric filters** turn log lines (for example "ERROR") into metrics.
- **Logs Insights:** query language to search and aggregate logs quickly.
- **Dashboards:** one view per service or environment; **Container Insights** for ECS/EKS, **Synthetics** canaries for URLs.

Good alarm practice: alarm on **symptoms** (latency, errors) first, use **percentiles (p95/p99)**, require several datapoints to avoid noise, and send to SNS -> Teams/Slack/pager.

Example:

```
# Logs Insights: top 5XX paths in the last hour
fields @timestamp, path, status
| filter status >= 500
| stats count(*) as errors by path
| sort errors desc
| limit 10
```

```
aws cloudwatch put-metric-alarm --alarm-name rds-orders-low-storage \
  --namespace AWS/RDS --metric-name FreeStorageSpace \
  --dimensions Name=DBInstanceIdentifier,Value=orders-prod \
  --statistic Average --period 300 --evaluation-periods 3 \
  --threshold 10737418240 --comparison-operator LessThanThreshold \
  --alarm-actions arn:aws:sns:ap-south-1:111111111111:db-alerts
```

> ***What to say to the interviewer:*** *For EC2 I watch CPU, credit balance, status checks and memory/disk through the CloudWatch agent; for ECS, service CPU and memory, running vs desired tasks and ALB errors and latency; for RDS, CPU, connections, free storage and memory, latency, IOPS and replica lag. I combine metrics, log groups with metric filters, Logs Insights queries and*

dashboards, and alarms use multiple datapoints and route through SNS to our Teams channels.

Q45. How do you optimize AWS cost for EC2, NAT, and S3?

Also asked as: What approaches do you use to reduce AWS costs?

In simple words: Stop paying for lights in empty rooms, buy in bulk what you always use, use discount seats for flexible trips, and move old boxes to cheaper storage.

Detailed answer

EC2:

- **Rightsize** with Compute Optimizer; move to **Graviton** (about 20 percent cheaper).
- **Savings Plans / Reserved Instances** for steady baseline; **Spot** for stateless, batch, CI, and EKS worker pools.
- **Schedule** dev/test off nights and weekends; delete idle instances, unattached EBS, old snapshots, unused EIPs; gp2 -> **gp3**.

NAT Gateway (charged per hour + per GB processed):

- Add **S3 and DynamoDB gateway endpoints** (free) so that traffic skips NAT.
- Interface endpoints for ECR, CloudWatch Logs when volume is high.
- Keep traffic in the same AZ (avoid cross-AZ data charges); find top talkers with VPC Flow Logs.

S3:

- Lifecycle to IA/Glacier, **Intelligent-Tiering** for unknown patterns, expire noncurrent versions, abort incomplete multipart uploads; use **Storage Lens**.

Governance: tagging, Cost Explorer, Budgets, **Cost Anomaly Detection**.

Example:

```
aws ec2 describe-volumes --filters Name=status,Values=available \
  --query "Volumes[].{ID:VolumeId,GB:Size}" --output table   # orphaned EBS

aws ce get-cost-and-usage --time-period Start=2026-09-01,End=2026-09-24 \
  --granularity MONTHLY --metrics UnblendedCost \
  --group-by Type=DIMENSION,Key=USAGE_TYPE
```

What to say to the interviewer: *For EC2 I rightsize, move to Graviton, cover the baseline with Savings Plans and use Spot for flexible workloads, while shutting down non-prod off-hours. For NAT I add free S3 and DynamoDB gateway endpoints, and for S3 I use lifecycle rules and Intelligent-Tiering, all tracked with tags, Budgets and Cost Anomaly Detection.*

Q46. What are best practices to protect an AWS-hosted web application from attacks?

In simple words: Layers like a castle: a moat (Shield), guards at the gate checking IDs (WAF), locked inner doors (private subnets and security groups), and cameras everywhere (GuardDuty, logs).

Detailed answer

1. **Edge:** CloudFront + **AWS WAF** (managed rules for OWASP Top 10, SQL injection, XSS, bad bots, rate-based rules, geo blocking) + **Shield** Standard (free) or Advanced (DDoS response team and cost protection).
2. **Network:** ALB in public subnets, app and DB in private subnets; SGs allow only what is needed; no SSH from internet (use SSM).
3. **TLS everywhere** with ACM; HSTS; redirect HTTP -> HTTPS.
4. **Identity:** least-privilege IAM roles, MFA, no root usage, no long-lived keys.
5. **Data:** KMS encryption at rest, Secrets Manager for credentials, private S3 with OAC.
6. **Patching and images:** hardened AMIs, Inspector scanning, ECR image scanning.
7. **Detection:** GuardDuty, Security Hub, CloudTrail, Config, VPC Flow Logs, WAF logs.
8. **App level:** input validation, authentication (Cognito), security headers, dependency scanning.
9. **Resilience:** autoscaling to absorb load, backups, incident runbooks.

Example:

```
User -> Route53 -> CloudFront (Shield) -> WAF (OWASP, rate 2000/5min/IP)
     -> ALB (443 only, ACM) -> App in private subnets (SG from ALB only)
     -> RDS in isolated subnets (SG from app only, KMS encrypted)
GuardDuty + Security Hub + CloudTrail watching all accounts
```

> ***What to say to the interviewer:*** *I use defense in depth: CloudFront with WAF managed and rate-based rules and Shield at the edge, private subnets with tight security groups, TLS everywhere, least-privilege IAM and encrypted data. Then I detect with GuardDuty, Security Hub and CloudTrail and keep images patched and scanned.*

Q47. Can you write a custom IAM policy? Explain its structure and how AWS evaluates policies.

> **In simple words:** An IAM policy is a permission slip: who may do which actions, on which things, under which conditions. If any slip anywhere says "NO", the answer is no.

Detailed answer

Structure of a statement:

- **Effect:** `Allow` or `Deny`.
- **Action:** API calls, for example `s3:GetObject`.
- **Resource:** ARNs the actions apply to (scope as narrowly as possible).
- **Condition:** extra rules, for example source VPC endpoint, MFA present, tag match, TLS only.
- **Principal:** only in resource-based policies (bucket policy, trust policy).

Managed vs inline:

- **AWS managed:** written by AWS, broad, convenient (for example `ReadOnlyAccess`).
- **Customer managed:** your reusable, versioned policies (preferred).
- **Inline:** embedded in one user/role, deleted with it; use for strict one-to-one cases.

Evaluation logic:

1. Everything starts as **implicit deny**.
2. An **explicit Deny** anywhere (SCP, permission boundary, identity, resource, session policy) **always wins**.
3. Otherwise an **Allow** is needed, and it must be allowed by every applicable layer (SCP and permission boundary must also allow).
4. Cross-account: both the identity policy and the resource policy must allow.

Example (least privilege: read/write only one prefix, only through our VPC endpoint):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListOwnPrefix",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::app-data",
      "Condition": { "StringLike": { "s3:prefix": ["orders/*"] } }
    },
    {
      "Sid": "ReadWriteOwnPrefix",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::app-data/orders/*",
      "Condition": { "StringEquals": { "aws:SourceVpce": "vpce-0abc123" } }
    }
  ]
}
```

```
aws iam create-policy --policy-name orders-s3-rw --policy-document file://policy.json
aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::111111111111:role/orders \
  --action-names s3:DeleteObject --resource-arns arn:aws:s3:::app-data/orders/x
```

> ***What to say to the interviewer:*** *Yes, a policy has statements with Effect, Action, Resource and optional Condition, and I write customer-managed policies scoped to exact ARNs and prefixes with conditions like source VPC endpoint. AWS starts from implicit deny, an explicit deny anywhere always wins, and an action is allowed only if every layer such as SCPs and boundaries also allows it; I test with the IAM policy simulator.*

Q48. Two employees work different shifts (10 AM-5 PM and 6 PM-2 AM). How do you give them AWS access based on time?

> **In simple words:** IAM can say "this pass is valid until 5 PM on 3rd October", but it cannot say "valid every day from 10 to 5". For a daily schedule you need a small robot that hands out and takes back the pass on time.

Detailed answer

Important limitation: the condition keys `aws:CurrentTime` and `aws:EpochTime` compare against an **absolute date-time**. IAM has **no native recurring daily time-window** condition, so a single static policy cannot express "10:00-17:00 every day".

Practical options:

1. **Scheduled automation:** EventBridge Scheduler triggers a Lambda at shift start to attach a policy (or create an IAM Identity Center **permission-set assignment**) and at shift end to detach/remove it. Handle time zones carefully (IST 10:00 = 04:30 UTC).
2. **Time-bounded policy renewed daily:** the Lambda rewrites the policy each day with that day's `DateGreaterThan`/`DateLessThan` window.
3. **Temporary elevated access** (for example AWS TEAM with IAM Identity Center): users request access for their shift, it is approved and auto-expires.
4. **Short session duration** on the permission set / role (for example 7-8 hours) so sessions do not outlive the shift. Note that revoking a policy does not kill an already-issued session, so short sessions matter.
5. Audit with CloudTrail; alert on off-shift activity.

Example (one day's window for the 10 AM-5 PM IST user, written in UTC):

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["ec2:Describe*", "cloudwatch:Get*", "logs:*"],
    "Resource": "*",
    "Condition": {
      "DateGreaterThan": { "aws:CurrentTime": "2026-10-01T04:30:00Z" },
      "DateLessThan":    { "aws:CurrentTime": "2026-10-01T11:30:00Z" }
    }
  }]
}
```

```
# EventBridge Scheduler (IST) calling a Lambda that grants/revokes access
aws scheduler create-schedule --name grant-shift-a \
  --schedule-expression "cron(0 10 * * ? *)" \
  --schedule-expression-timezone Asia/Kolkata \
  --flexible-time-window Mode=OFF \
  --target file://grant-shift-a-target.json
```

> ***What to say to the interviewer:*** *IAM's aws:CurrentTime condition only compares against fixed dates, so there is no built-in daily recurring time window. I would use EventBridge Scheduler with a Lambda to grant and revoke the permission-set assignment at each shift boundary, or a just-in-time access tool like TEAM, with short session durations and CloudTrail alerts for off-shift activity.*

Q49. How do you connect AWS to an on-premises data center?

> **In simple words:** Either build an encrypted tunnel over the public internet (VPN - quick and cheap), or lay a private dedicated cable into AWS (Direct Connect - fast and steady).

Detailed answer

Option	What	When
Site-to-Site VPN	Two IPsec tunnels over the internet to a VGW or TGW	Quick setup, backup link, lower bandwidth
Direct Connect (DX)	Dedicated private link 1/10/100 Gbps via a DX location	Steady high bandwidth, low latency, lower data-out cost
DX + VPN backup	Primary DX, VPN failover	Production hybrid
Transit Gateway	Hub attaching VPN/DX to many VPCs	Many VPCs/accounts
Client VPN	Per-user OpenVPN access	Remote engineers

Other pieces:

- **BGP** for dynamic routing; non-overlapping CIDRs between on-prem and VPCs.
- **Hybrid DNS:** Route 53 Resolver **inbound endpoints** (on-prem resolves AWS private zones) and **outbound endpoints + forwarding rules** (AWS resolves `corp.local`).
- Security Groups/NACLs allow on-prem ranges; firewall rules on-prem.
- For migrations: Storage Gateway, DataSync, DMS, Application Migration Service.

Example:

```
On-prem DC (192.168.0.0/16, BGP router)
   ||== Direct Connect (primary) ==||
   ||-- Site-to-Site VPN (backup) --||
               Transit Gateway
            /        |         \
        prod VPC   test VPC   shared VPC (Route 53 Resolver in/out endpoints)
```

> ***What to say to the interviewer:*** *I connect on-premises through a Site-to-Site VPN for quick or backup connectivity and Direct Connect for dedicated high bandwidth, usually terminated on a Transit Gateway so all VPCs can use it with BGP routing. For DNS I use Route 53 Resolver inbound and outbound endpoints, and Client VPN for individual users.*

Q50. How do you manage and connect to services like EC2, RDS, EKS and ECS?

> **In simple words:** Every service has its own secure front door, and all of them open with your AWS identity instead of shared passwords or open ports.

Detailed answer

- **EC2:** SSM Session Manager (no SSH port, no keys, logged).
- **EKS:** `aws eks update-kubeconfig` writes a kubeconfig that authenticates with your IAM identity (mapped via EKS access entries); then use `kubectl`.
- **ECS: ECS Exec** opens a shell in a running container. Needs `enableExecuteCommand` on the service/task, the SSM agent (ECS injects it automatically), and the **task role** allowing the `ssmmessages` channel actions.
- **RDS:** SSM port forwarding through an instance in the VPC, then a normal DB client to `localhost`.
- Everything else managed through **IaC** (Terraform) and the CLI, with IAM Identity Center for human access.

Example:

```
# EC2
aws ssm start-session --target i-0abc123def4567890

# EKS
aws eks update-kubeconfig --name prod-eks --region ap-south-1
kubectl get nodes

# ECS Exec
aws ecs update-service --cluster prod --service orders \
  --enable-execute-command --force-new-deployment
aws ecs execute-command --cluster prod --task 0f1e2d3c4b5a \
  --container orders --interactive --command "/bin/sh"

# RDS through SSM port forwarding
aws ssm start-session --target i-0abc123def4567890 \
  --document-name AWS-StartPortForwardingSessionToRemoteHost \
  --parameters '{"host":["orders.abc123.ap-south-1.rds.amazonaws.com"],
                 "portNumber":["5432"],"localPortNumber":["15432"]}'
psql -h localhost -p 15432 -U app_ro orders
```

> ***What to say to the interviewer:*** *I use IAM-based access everywhere: SSM Session Manager for EC2, aws eks update-kubeconfig plus kubectl for EKS, ECS Exec with enableExecuteCommand and the right task role for containers, and SSM port forwarding to reach private RDS. Nothing needs public IPs, open SSH ports or shared keys, and all sessions are logged.*

Q51. How do you establish database connections from your applications in deployments?

> **In simple words:** The app looks up the database address and password in a safe (Secrets Manager), travels only on private roads (private subnets, security groups), and shares a few taxis (a connection pool) instead of calling a new one for every trip.

Detailed answer

1. **Endpoint and credentials** come from config/Secrets Manager (via External Secrets on EKS or ECS task `secrets`), never hard-coded in the image or Git.
2. **Network:** DB in private/data subnets, no public access; the DB security group allows 5432/3306 **only from the app security group** (SG-to-SG reference).
3. **Connection pooling** in the app (HikariCP, pgbouncer, SQLAlchemy pool) and **RDS Proxy** for spiky or Lambda workloads.
4. **Authentication:** Secrets Manager with rotation, or **IAM database authentication** tokens via IRSA/task role.
5. **TLS** in transit (`sslmode=require` or `verify-full` with the RDS CA bundle); KMS encryption at rest.
6. **Resilience:** use the DNS endpoint (not IP), retries with backoff, a separate reader endpoint for read traffic.

Example:

```
# EKS Deployment snippet
env:
  - name: DB_HOST
    value: orders-proxy.proxy-abc123.ap-south-1.rds.amazonaws.com
  - name: DB_PASSWORD
    valueFrom: { secretKeyRef: { name: orders-db, key: password } }
  - name: DB_SSLMODE
    value: verify-full
```

```
aws ec2 authorize-security-group-ingress --group-id sg-rds \
  --protocol tcp --port 5432 --source-group sg-app
```

> ***What to say to the interviewer:*** *The app gets the DB endpoint and credentials from Secrets Manager, connects over TLS from private subnets, and the database security group only accepts traffic from the app's security group. I use connection pooling or RDS Proxy, rotate credentials or use IAM auth tokens, and always connect by DNS endpoint with retries.*

Q52. How do you create AWS Lambda functions and manage their deployment artifacts?

> **In simple words:** Package your code like a parcel (a zip or a container image), store it in a warehouse (S3 or ECR), label every version, and use a pointer (alias) that you can slowly move to the new parcel and move back if it breaks.

Detailed answer

Packaging:

- **Zip package** (50 MB direct upload, 250 MB unzipped) uploaded to **S3** with a versioned key, for example `lambda/orders/1.4.2.zip`.
- **Container image** (up to 10 GB) pushed to **ECR**, tagged with the git SHA.
- **Layers** for shared dependencies/libraries across functions.

Tooling: AWS SAM, Serverless Framework, CDK or Terraform; CI builds, tests and uploads the artifact, then updates the function.

Versions and aliases:

- Publishing creates an immutable **version**.
- An **alias** (`live`, `prod`) points to a version; clients invoke the alias.
- **Canary with CodeDeploy:** weighted alias shifts, for example 10 percent for 5 minutes then 100 percent, with automatic rollback on CloudWatch alarms (SAM `DeploymentPreference`).

Example:

```
aws s3 cp build/orders-1.4.2.zip s3://artifacts-111111111111/lambda/orders/1.4.2.zip
aws lambda update-function-code --function-name orders \
  --s3-bucket artifacts-111111111111 --s3-key lambda/orders/1.4.2.zip --publish
aws lambda update-alias --function-name orders --name live --function-version 7 \
  --routing-config '{"AdditionalVersionWeights":{"8":0.1}}'
```

```
# SAM template: automatic canary + rollback
OrdersFunction:
  Type: AWS::Serverless::Function
  Properties:
    CodeUri: src/
    Handler: app.handler
    Runtime: python3.12
    AutoPublishAlias: live
    DeploymentPreference:
      Type: Canary10Percent5Minutes
      Alarms: [!Ref OrdersErrorsAlarm]
```

What to say to the interviewer: *CI builds the function as a versioned zip in S3 or a container image in ECR, with shared libraries in layers, and deploys it with SAM or Terraform. Each release publishes an immutable version, clients call an alias, and CodeDeploy shifts the alias gradually, for example 10 percent for 5 minutes, rolling back automatically on alarms.*

Q53. What is the hub-and-spoke network model?

In simple words: Like an airline hub: instead of direct flights between every pair of small cities, all flights go through one big airport. Fewer routes, one place for security checks.

Detailed answer

- **Hub:** an **AWS Transit Gateway** (a regional router) that every VPC and on-prem link attaches to.
- **Spokes:** workload VPCs (prod, test, per-team) that only connect to the hub, not to each other directly.
- **Shared VPCs on the hub:**
- **Shared services VPC:** CI runners, AD/DNS, artifact repos, monitoring.
- **Egress VPC: centralized NAT Gateways** (and proxy) so spokes do not each pay for NAT.
- **Inspection VPC:** AWS Network Firewall or third-party appliances (via Gateway Load Balancer) for east-west and north-south traffic.
- **TGW route tables** create segmentation, for example prod spokes cannot reach dev spokes, but both reach shared services.
- **Versus a peering mesh:** N VPCs need N x (N-1) / 2 peerings (10 VPCs = 45), with no transitive routing and no central inspection. Hub-and-spoke scales to thousands of VPCs and one VPN/DX serves all.
- **Trade-offs:** TGW charges per attachment-hour and per GB, and adds a hop.
- It is the same idea as **Azure hub-spoke** (hub VNet with firewall/VPN gateway, peered spoke VNets) and GCP hub-and-spoke with Network Connectivity Center.

Example:

```
                On-prem (VPN / Direct Connect)
                          |
   Egress VPC (NAT) -- Transit Gateway -- Inspection VPC (Network Firewall)
                     /      |       \
               prod VPC  dev VPC  shared-services VPC
   TGW RT "prod": prod -> shared, egress, on-prem     (no route to dev)
   TGW RT "dev" : dev  -> shared, egress              (no route to prod)
```

What to say to the interviewer: *Hub-and-spoke puts a Transit Gateway at the center with workload VPCs as spokes, plus shared services, centralized egress NAT and an inspection VPC attached to the hub. TGW route tables segment prod from dev, and it scales far better than a peering mesh, which needs N times N minus 1 over 2 connections and has no transitive routing.*

Q54. If you do not want to use Auto Scaling policies, how else can you scale EC2?

In simple words: Instead of an automatic thermostat, you can use a timer (schedules), a helper who reacts to alarms (Lambda), a forecast (predictive scaling), or just turn the knob yourself (manual).

Detailed answer

1. **Scheduled actions** on the ASG: set min/max/desired at fixed times (for example scale to 20 before 9 AM, back to 4 at 9 PM).
2. **Predictive scaling:** ML forecasts daily/weekly patterns and scales ahead of time (can run in forecast-only mode first).
3. **CloudWatch alarm -> SNS/EventBridge -> Lambda:** your own logic calls `set-desired-capacity`, or launches/starts/stops standalone instances.
4. **EventBridge Scheduler** calling EC2 start/stop APIs directly (great for dev/test office hours).
5. **Manual:** `set-desired-capacity` from CLI/console or a CI job before an event.
6. **Vertical scaling:** stop, change instance type, start (causes downtime for that instance).

Important: **CloudWatch alarm EC2 actions** can only **stop, terminate, reboot or recover** an instance. They cannot add capacity, so scaling out always needs an ASG action, a Lambda, or you.

Example:

```
aws autoscaling put-scheduled-update-group-action --auto-scaling-group-name web-asg \
  --scheduled-action-name office-hours-up --recurrence "30 3 * * MON-FRI" \
  --min-size 6 --desired-capacity 10 --max-size 20          # 09:00 IST = 03:30 UTC

aws autoscaling set-desired-capacity --auto-scaling-group-name web-asg \
  --desired-capacity 15 --honor-cooldown

aws scheduler create-schedule --name stop-dev-nightly \
  --schedule-expression "cron(0 21 * * ? *)" --schedule-expression-timezone Asia/Kolkata \
  --flexible-time-window Mode=OFF --target file://stop-dev-target.json
```

What to say to the interviewer: *Without dynamic policies I can use ASG scheduled actions, predictive scaling, or a CloudWatch alarm that triggers a Lambda through SNS or EventBridge to call set-desired-capacity, plus EventBridge Scheduler to start and stop instances. Alarm EC2 actions can only stop, terminate, reboot or recover, so they cannot add capacity by themselves.*

Scenario-Based

Q55. An application behind an ALB is returning 503 errors. How would you troubleshoot it?

In simple words: 503 means the receptionist (ALB) is at the desk but has nobody to pass your request to. So first check whether any healthy staff (targets) are sitting behind the desk.

Detailed answer

First, who produced the error? Compare `HTTPCode_ELB_5XX_Count` (ALB generated it) with `HTTPCode_Target_5XX_Count` (your app returned it).

If the ALB returned 503:

1. **Target group has no registered targets** (ASG scaled to 0, ECS tasks all stopped, wrong target group on the listener rule).
2. **Targets unhealthy:** check `describe-target-health` reasons (`Target.Timeout`, `Target.ResponseCodeMismatch`, `Target.FailedHealthChecks`). Note: if all targets are unhealthy, ALB "fails open" and still sends traffic, so errors then look like 502/504 too.
3. **Health check misconfig:** wrong path/port, app returns 301/401 on `/health`, too short timeout.
4. **Security groups:** target SG does not allow the ALB SG on the app port.
5. **Listener rule** with a fixed-response 503 or forwarding to an empty group; Lambda target throttled.
6. **Deployments:** all old targets draining at once.

If the app returned 503: app overloaded, dependency (DB/Redis) down, thread pool exhausted - check app logs and ALB access logs (`target_status_code`).

Example:

```
aws elbv2 describe-target-health --target-group-arn arn:aws:elasticloadbalancing:...:tg/web/abc
# {"State":"unhealthy","Reason":"Target.ResponseCodeMismatch","Description":"...[404]"}

aws cloudwatch get-metric-statistics --namespace AWS/ApplicationELB \
  --metric-name HTTPCode_ELB_5XX_Count --dimensions Name=LoadBalancer,Value=app/web/123 \
  --start-time 2026-09-24T09:00:00Z --end-time 2026-09-24T10:00:00Z \
  --period 60 --statistics Sum
```

> ***What to say to the interviewer:*** *I first check whether the 503 comes from the ALB or the targets using the ELB vs Target 5XX metrics. For ALB 503s I check that the target group has registered, healthy targets, the health check path and security groups are correct and the listener rules point to the right group; for app 503s I go to application logs and dependencies.*

Q56. An EC2 instance behind an ALB becomes unreachable. How do you troubleshoot it in an existing infrastructure, and which metrics, tools and CLI commands do you use?

Also asked as: What metrics would you monitor for this setup? Which CLI commands would you run? An EC2 instance is unreachable and it is NOT a security group issue - what else do you check?

> **In simple words:** Walk the path the request takes: reception (ALB) -> corridor doors (security groups, NACLs) -> the office (instance) -> the person at the desk (the app). Find the first place where it stops.

Detailed answer

Step-by-step:

1. **Target group health:** `describe-target-health` gives the reason code:

- `Target.Timeout`: no answer - usually security group, NACL, or the app not listening.

- `Target.FailedHealthChecks`: connects but the check fails.
- `Target.ResponseCodeMismatch`: app returns, for example, 301/404/500 on the health path.
- `Target.NotInUse` / `Target.NotRegistered`: wrong AZ or not registered.

2. **EC2 status checks:** `describe-instance-status` - system check failed = AWS hardware (stop/start); instance check failed = OS (kernel, disk, memory).
3. **Security group chain:** the ALB SG must allow outbound to the instance, and the **instance SG must allow the app port from the ALB SG**.
4. **NACLs** on both subnets: app port inbound and **ephemeral ports 1024-65535** for return traffic.
5. **Health check config:** path, port, success codes (matcher), timeout and thresholds match the app.
6. **Inside the instance** (via SSM, no SSH needed): is the service running (`systemctl`), listening on the right port and interface (`ss -tlnp`, not bound only to 127.0.0.1 when using a different port), does `curl localhost:8080/health` work, is the **disk full** (`df -h`) or was there an **OOM kill** (`dmesg`)?
7. **What changed?** Recent deploy, new AMI, config or SG change (CloudTrail), certificate expiry.
8. **ASG behaviour:** the ASG may be replacing it (health check type ELB) - check scaling activities.
9. **ALB access logs:** `target_status_code`, `-` means no response; `elb_status_code` 502/504.

If the security groups are fine, check these next:

- **Status checks:** system check failed = AWS host or network problem (stop/start moves it to healthy hardware); instance check failed = OS problem.
- **Look without logging in:** `get-console-output` (system log) and `get-console-screenshot` show a kernel panic, an OOM kill, fsck waiting at boot, or a bad `/etc/fstab` entry.
- **Network path:** NACL rules (inbound port and outbound ephemeral ports), the subnet **route table** (a deleted or changed route to the IGW/NAT/TGW), and a changed **Elastic IP or ENI** (EIP moved to another instance, secondary ENI detached, private IP changed so the target registration is stale).
- **Resource exhaustion:** a full root disk stops services and logins; memory exhaustion causes OOM kills or a hang.
- **SSM:** if the SSM agent still responds, use Session Manager or Run Command to inspect it without any SSH; if SSM is also offline, the OS is probably hung, so reboot, or detach the root volume and fix it on a rescue instance.

Metrics to monitor for this setup:

Where	Metrics
ALB / target group	HealthyHostCount, UnHealthyHostCount, HTTPCode_ELB_5XX_Count vs HTTPCode_Target_5XX_Count, TargetResponseTime (p99), RequestCount, RejectedConnectionCount

Where	Metrics
EC2	CPUUtilization, StatusCheckFailed_System / _Instance, NetworkIn/Out, CPUCreditBalance
CloudWatch Agent	mem_used_percent, disk_used_percent, process counts
Logs	App logs and ALB access logs in CloudWatch Logs / S3 (Athena)

Example:

```
aws elbv2 describe-target-health --target-group-arn <tg-arn> \
  --query "TargetHealthDescriptions[].{id:Target.Id,state:TargetHealth.State,
  reason:TargetHealth.Reason}"
aws ec2 describe-instance-status --instance-ids i-0abc --include-all-instances
aws ec2 describe-security-groups --group-ids sg-app \
  --query "SecurityGroups[0].IpPermissions"
aws elbv2 describe-target-groups --target-group-arns <tg-arn> \
  --query "TargetGroups[0].{path:HealthCheckPath,port:HealthCheckPort,codes:Matcher}"
aws ssm start-session --target i-0abc
# on the instance:
sudo systemctl status myapp
sudo ss -tlnp | grep 8080
curl -i http://localhost:8080/health
df -h ; free -m ; sudo dmesg | grep -i -E "killed process|out of memory"
```

What to say to the interviewer: *I start with describe-target-health to get the reason code, then check the EC2 status checks, the security group chain from the ALB SG to the instance port, NACLs and the health check settings. Then I get in with SSM to confirm the service is running and listening, curl the health path locally and check disk and OOM, while correlating with recent deploys; I monitor HealthyHostCount, ELB versus target 5XX, TargetResponseTime, status checks and agent memory and disk metrics.*

Q57. What is the typical latency of a load balancer, and how do you troubleshoot high latency on an ALB?

Also asked as: Explain how you would troubleshoot high latency in an ALB.

In simple words: The receptionist itself is quick (a few milliseconds). If the queue is slow, it is almost always the staff behind the desk (your targets) or the road to them.

Detailed answer

- ALB itself normally adds only **single-digit milliseconds**. High latency almost always comes from targets or dependencies.
- `TargetResponseTime` = time from ALB sending the request to the target starting to respond. Look at **p95/p99**, not only average.

Triage order:

1. **CloudWatch:** TargetResponseTime (p99), RequestCount, HTTPCode_Target_5XX, ActiveConnectionCount, RejectedConnectionCount, UnHealthyHostCount. Is it all targets or one AZ/target?
2. **ALB access logs:** split `request_processing_time`, `target_processing_time`, `response_processing_time`. High target time -> app; high request/response time -> ALB/client network.

3. **Targets:** CPU, memory, CPU credits (T instances), GC pauses, thread/connection pools, too few targets (scaling lag).
4. **Dependencies:** RDS latency/connections, Redis, external APIs; use **X-Ray/OpenTelemetry** traces to find the slow hop.
5. **Config:** cross-zone disabled with uneven targets, sticky sessions overloading one target, slow start, keep-alive mismatch (idle timeout).
6. **Client side:** far-away users -> add CloudFront.

Example:

```
# Athena query on ALB access logs: slowest targets in the last hour
SELECT target_ip, avg(target_processing_time) AS avg_s,
       approx_percentile(target_processing_time, 0.99) AS p99_s, count(*) AS reqs
FROM alb_logs
WHERE time > date_add('hour', -1, now())
GROUP BY target_ip ORDER BY p99_s DESC LIMIT 10;
```

> ***What to say to the interviewer:*** *The ALB itself adds only a few milliseconds, so I look at TargetResponseTime p99 and the access log fields to separate ALB, target and network time. Then I check target CPU, memory and pools, dependencies like RDS through traces, and configuration issues like sticky sessions or disabled cross-zone balancing.*

Q58. Users are reporting slow application performance. Which AWS services and metrics would you check first?

Also asked as: A service is slow - how do you debug performance issues in AWS?

> **In simple words:** Follow the request's journey from the user's door to the database and back, and time every stop. The slow stop is your culprit.

Detailed answer

Step-by-step, outside-in:

1. **Scope:** all users or some? All endpoints or one? When did it start? Any deployment or config change?
2. **Edge/CDN:** CloudFront cache hit ratio, origin latency.
3. **Load balancer:** ALB `TargetResponseTime` p99, 5XX, request count (traffic spike?).
4. **Compute:** EC2/ECS/EKS CPU, memory, **CPUCreditBalance** (T-series throttling is a classic), network, pod restarts, throttling from CPU limits.
5. **Storage:** EBS `VolumeQueueLength`, `BurstBalance` (gp2 burst exhaustion), IOPS limits.
6. **Database:** RDS CPU, `DatabaseConnections`, Read/WriteLatency, `DiskQueueDepth`, locks; **Performance Insights** shows top SQL and wait events.
7. **Cache/queues:** ElastiCache CPU, evictions, latency; SQS age of oldest message; MSK consumer lag.
8. **Traces:** X-Ray / OpenTelemetry to see which service or call is slow.
9. **Logs:** CloudWatch Logs Insights for slow requests and errors.
10. **External:** third-party API latency, AWS Health Dashboard.

Example:

```
Deploy at 14:05 -> p99 latency 180 ms -> 2.4 s
Performance Insights: top wait = IO:DataFileRead, query "SELECT ... FROM orders WHERE email=?"
Root cause: new release dropped an index in migration. Fix: recreate index, p99 back to 190 ms.
```

What to say to the interviewer: *I scope the problem first, then go outside-in: CloudFront, ALB TargetResponseTime, compute CPU, memory and credit balance, EBS burst balance, then RDS with Performance Insights and caches, using X-Ray traces to pinpoint the slow hop. I always correlate the start time with recent deployments or config changes.*

Q59. Two users access the same application: one sees low latency and the other sees high latency. How do you troubleshoot?

In simple words: Same shop, two customers, different waiting times. Maybe one lives far away, came by a jammed road, got sent to a slow counter, or ordered something much bigger.

Detailed answer

Compare the two users on each layer:

1. **Geography and edge:** where are they? Which **CloudFront edge** served them (`x-amz-cf-pop` header)? Cache hit (`x-cache: Hit from cloudfront`) for one and miss for the other?
2. **DNS routing:** Route 53 latency/geolocation rules may send them to different regions; check what `dig` returns for each.
3. **Network path / ISP:** `traceroute`/`mtr` from the slow user; packet loss, VPN or corporate proxy, Wi-Fi.
4. **Which target/AZ:** ALB access logs by client IP show `target_ip` and `target_processing_time`. **Sticky sessions** may pin the slow user to an overloaded or unhealthy target.
5. **User-specific data:** bigger account, more records, heavier queries, different feature flags or permissions (for example an admin dashboard loading everything).
6. **Client device/browser:** old device, extensions, browser dev tools timing (DNS, TLS, TTFB, download).
7. **Traces:** filter X-Ray/OTel by user ID to see the slow span.

Example:

```
curl -s -o /dev/null \
  -w "dns=%{time_namelookup} tls=%{time_appconnect} ttfb=%{time_starttransfer}\n" \
  https://app.example.com/api/dashboard
curl -sI https://app.example.com/ | grep -i -E "x-cache|x-amz-cf-pop"
mtr -rwc 50 app.example.com
```

What to say to the interviewer: *I compare the two users layer by layer: their location and CloudFront edge or cache status, where Route 53 sends them, and their network path with traceroute or mtr. Then I use ALB access logs and traces filtered by their client IP or user ID to see whether sticky sessions pinned them to a slow target or whether their data makes queries heavier.*

Q60. An Auto Scaling Group is not launching new instances during traffic spikes. How would you investigate?

Also asked as: Explain a real scenario where Auto Scaling did not scale properly. What was the root cause?

> **In simple words:** The manager wants to hire but nobody shows up. Either the manager never got the message (alarm), the hiring budget is maxed out (max size/quotas), or the job ad is broken (launch template).

Detailed answer

Check in this order:

1. **Scaling activity history:** `describe-scaling-activities` shows the exact failure message.
2. **Capacity limits:** desired already equals **MaxSize**.
3. **Alarm/policy:** did the CloudWatch alarm fire? Wrong metric, too long period, or scaling on CPU when the bottleneck is memory or requests. Cooldown/instance warmup delaying action.
4. **Suspended processes:** `Launch` or `AlarmNotification` suspended (someone forgot to resume after maintenance).
5. **Launch template errors:** deleted AMI, missing instance profile/KMS permission, invalid SG/subnet, wrong key pair.
6. **Capacity and quotas:** `InsufficientInstanceCapacity` in the AZ, **EC2 vCPU service quota** reached, Spot capacity gone, subnet out of IP addresses.
7. **Health checks:** new instances launch but fail ELB health check and get terminated (launch-terminate loop).

Real example: during a sale, the ASG hit its vCPU quota for the instance family; activities showed `VcpuLimitExceeded`. Fix: raised the quota, added multiple instance types across 3 AZs in a mixed instances policy, and moved to target tracking on `ALBRequestCountPerTarget` with scheduled/predictive scaling before known peaks.

Example:

```
aws autoscaling describe-scaling-activities --auto-scaling-group-name web-asg --max-items 5
aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names web-asg \
  --query "AutoScalingGroups[0].{Min:MinSize,Max:MaxSize,Desired:DesiredCapacity,
  Suspended:SuspendedProcesses}"
aws service-quotas get-service-quota --service-code ec2 --quota-code L-1216C47A
```

> ***What to say to the interviewer:*** *I start with the ASG scaling activity history, which usually states the reason, then check max size, whether the alarm and policy actually fired, suspended processes, launch template errors, AZ capacity, vCPU quotas and subnet IPs. In one case we hit the vCPU quota, so we raised it, diversified instance types and added predictive scaling on requests per target.*

Q61. Your EC2 instances scale up but never scale down. How would you debug and fix it?

Also asked as: An Auto Scaling Group scales out but not in.

> **In simple words:** Extra staff were hired for the rush but nobody was sent home afterwards. Either the "it is quiet now" signal never comes, or someone blocked the exit door.

Detailed answer

Common causes:

1. **Metric never drops:** the scale-in alarm threshold is too low, or background jobs keep CPU up. Memory-based scaling rarely drops because apps keep memory allocated.
2. **Target tracking scale-in is conservative** (needs a sustained low period) or `DisableScaleIn` is true.
3. **Only a scale-out policy exists** (step scaling without a scale-in policy).
4. **MinSize or desired manually raised** (MinSize = current count).
5. **Scale-in protection** on instances, or the `Terminate` process suspended.
6. **Lifecycle hooks** stuck in `Terminating:Wait` (hook never completes).
7. **Conflicting policies** (a scheduled action resets desired capacity; multiple policies where scale-out keeps winning).
8. On ECS/EKS: the node cannot be drained (PodDisruptionBudgets, pods with local storage, `do-not-disrupt` annotations).

Fix: correct metric and thresholds (requests per target is better than CPU for web), ensure a scale-in path exists, remove unneeded protection, complete lifecycle hooks with timeouts, and alarm on "desired > baseline for 6 hours".

Example:

```
aws autoscaling describe-policies --auto-scaling-group-name web-asg
aws autoscaling describe-auto-scaling-instances \
  --query "AutoScalingInstances[].{ID:InstanceId,Protected:ProtectedFromScaleIn,
  State:LifecycleState}"
aws autoscaling set-instance-protection --auto-scaling-group-name web-asg \
  --instance-ids i-0abc --no-protected-from-scale-in
```

> ***What to say to the interviewer:*** *I check whether the scale-in metric ever crosses its threshold, whether a scale-in policy even exists or target tracking has scale-in disabled, and whether MinSize, scale-in protection, suspended Terminate processes or stuck lifecycle hooks are blocking it. Then I fix the metric choice and thresholds and add an alarm if capacity stays above baseline for hours.*

Q62. An RDS database becomes unavailable during business hours. What is your recovery approach?

> **In simple words:** The shop's cash counter stops. First, open a backup counter fast (restore service), keep customers informed, and only after the rush find out why it broke.

Detailed answer

1. Triage (first 5 minutes):

- `describe-db-instances` status: `storage-full`, `rebooting`, `failing-over`, `maintenance`, `incompatible-parameters`, `inaccessible-encryption-credentials`?
- **RDS Events** and the **AWS Health Dashboard** (AZ issue?).
- CloudWatch: FreeStorageSpace, CPU, DatabaseConnections (max connections reached looks like "down" to apps), FreeableMemory.

- Is it the DB or the network (SG change, DNS, secret rotated)?

2. Restore service:

- **Multi-AZ:** failover is automatic; if hung, `reboot --force-failover`.
- **storage-full:** increase allocated storage (enable storage autoscaling).
- **Connections exhausted:** kill idle sessions, scale app pools down, add RDS Proxy.
- **Single-AZ and broken:** promote a read replica or **point-in-time restore** to a new instance, then repoint the app (DNS CNAME / secret).

3. Communicate status every 15-30 minutes; put the app in maintenance mode if needed.

4. After: RCA, then prevent: Multi-AZ, storage autoscaling, alarms on storage/connections, maintenance windows outside business hours, tested restore runbook.

Example:

```
aws rds describe-events --source-identifier orders-prod --source-type db-instance \
  --duration 120
aws rds modify-db-instance --db-instance-identifier orders-prod \
  --allocated-storage 500 --max-allocated-storage 1000 --apply-immediately
aws rds restore-db-instance-to-point-in-time --source-db-instance-identifier orders-prod \
  --target-db-instance-identifier orders-restored --restore-time 2026-09-24T09:40:00Z
```

> ***What to say to the interviewer:*** *I check the instance status, RDS events and key metrics like free storage and connections to find why it is down, then restore service fastest: failover for Multi-AZ, add storage if full, or promote a replica or do a point-in-time restore and repoint the app. I keep stakeholders updated and follow with an RCA and preventive alarms, Multi-AZ and storage autoscaling.*

Q63. An RDS instance is hitting max connections during peak traffic. How would you handle it?

Also asked as: RDS connections suddenly spike - how do you monitor and protect the database without downtime?

> **In simple words:** The restaurant has 100 chairs and every waiter is holding chairs for guests who left. Share chairs better (pooling) before building a bigger restaurant.

Detailed answer

Monitor: CloudWatch `DatabaseConnections` (alarm at 80 percent of `max_connections`), **Performance Insights** (sessions by wait event, top SQL, top hosts/users), and the app's pool metrics. A sudden jump right after a deploy usually means a connection leak or a bigger pool setting.

Immediate (no downtime):

1. Find who holds connections: `pg_stat_activity` / `SHOW PROCESSLIST` grouped by user, application and client address to find the **leaking client**; kill long-idle sessions.
2. Reduce per-pod/instance **pool sizes**; many small pods each with a pool of 50 add up fast (20 pods x 50 = 1,000).

Short term:

- **RDS Proxy** pools and multiplexes connections, and helps greatly with Lambda.
- **pgbouncer** in transaction mode for PostgreSQL.

- Route reads to **read replicas** (reader endpoint).
- Add caching (ElastiCache) for hot reads.

Longer term:

- `max_connections` is derived from instance memory; raising it or moving to a bigger class helps but costs memory per connection.
- Fix connection leaks, set idle timeouts, cap autoscaling so pods x pool <= DB limit.
- Alarm at 80 percent of `DatabaseConnections`.

Example:

```
SELECT usename, application_name, client_addr, state, count(*)
FROM pg_stat_activity GROUP BY 1,2,3,4 ORDER BY 5 DESC;

SELECT pg_terminate_backend(pid) FROM pg_stat_activity
WHERE state = 'idle' AND state_change < now() - interval '10 minutes';
```

> ***What to say to the interviewer:*** *I watch DatabaseConnections and Performance Insights, and when it spikes I use pg_stat_activity grouped by application and client to find the leaking client, then trim idle sessions and oversized pools, then put RDS Proxy or pgbouncer in front to multiplex connections and move reads to replicas and cache. Long term I size pools so pods times pool size stays under the limit, fix leaks, and alarm at 80 percent of max connections.*

Q64. Your ECS service is stuck in PROVISIONING. What steps would you take?

> **In simple words:** The dish is ordered but there is no free stove to cook it on. ECS is waiting for capacity or a network slot before the task can start.

Detailed answer

`PROVISIONING` means ECS is waiting for resources before the task can run: capacity to be added, or the network interface to be attached.

Checks:

1. **Service events:** `describe-services` -> `events` often says "unable to place a task" and why.
2. **EC2 capacity provider:** managed scaling must scale the ASG; check ASG **max size**, scaling activities, and `CapacityProviderReservation` metric. The ASG may be at max or failing to launch.
3. **Resources:** no instance has enough free CPU/memory/ports for the task size; placement constraints too strict.
4. **awsvpc networking:** subnets out of **free IP addresses**; ENI limit per instance reached (enable ENI trunking).
5. **Fargate:** capacity shortage in the AZ (spread across subnets in several AZs), Fargate Spot unavailable, Fargate vCPU quota.
6. **Task launch errors** after provisioning: image pull (ECR permissions, no NAT/endpoints), secrets fetch failure -> check stopped task `stoppedReason`.

Example:

```
aws ecs describe-services --cluster prod --services orders \
  --query "services[0].events[:5].message"
aws ecs describe-tasks --cluster prod --tasks <task-id> \
  --query "tasks[0].{status:lastStatus,reason:stoppedReason}"
aws ec2 describe-subnets --subnet-ids subnet-a subnet-b \
  --query "Subnets[].{id:SubnetId,free:AvailableIpAddressCount}"
```

What to say to the interviewer: *PROVISIONING means ECS is waiting for capacity or networking, so I read the service events first, then check the capacity provider's ASG max size and scaling activity, free CPU and memory, subnet IPs and ENI limits for awsvpc. For Fargate I check AZ capacity and quotas, and if tasks then stop I look at stoppedReason for image pull or secrets issues.*

Q65. Describe an incident where an EC2 instance crashed or became unresponsive. What did you do?

In simple words: When a machine stops answering, first find out whether it is the building (AWS host) or the machine itself (your OS). Get it working again fast, then do the post-mortem.

Detailed answer

Structured approach:

1. **Status checks:** `StatusCheckFailed_System` = AWS host problem -> stop/start moves it to new hardware. `StatusCheckFailed_Instance` = OS problem (kernel panic, OOM, disk full, bad fstab).
2. **Look without logging in:** `get-console-output` and **instance screenshot** for kernel panic/OOM messages.
3. **Metrics before the crash:** CPU, memory and disk from the CloudWatch agent, CPU credits.
4. **Recover:** reboot; if it will not boot, stop, detach the root volume, attach to a rescue instance, fix (fstab, full disk), reattach. Or restore from the latest snapshot/AMI.
5. **Prevent:** EC2 auto-recovery alarm, run behind an ASG so it self-heals, disk/memory alarms, log rotation.

Sample real story (adapt to yours): "Our Zabbix monitoring server's web UI started returning 504. The EC2 was hung at OS level, so SSM and SSH did not respond. I checked the console output, rebooted, verified the web server and the `mysqld` service came back, and confirmed monitoring resumed. RCA found memory and disk pressure on a small instance, so we tuned PHP-FPM workers, vacuumed the systemd journal from 91 percent to 82 percent disk, and added disk and memory alarms."

Example:

```
aws ec2 describe-instance-status --instance-ids i-0abc --include-all-instances
aws ec2 get-console-output --instance-id i-0abc --latest --output text | tail -50
aws ec2 get-console-screenshot --instance-id i-0abc --query ImageData --output text \
  | base64 -d > screen.jpg
aws cloudwatch put-metric-alarm --alarm-name ec2-autorecover-i-0abc \
  --namespace AWS/EC2 --metric-name StatusCheckFailed_System --statistic Maximum \
  --dimensions Name=InstanceId,Value=i-0abc --period 60 --evaluation-periods 2 \
  --threshold 1 --comparison-operator GreaterThanOrEqualToThreshold \
  --alarm-actions arn:aws:automate:ap-south-1:ec2:recover
```

What to say to the interviewer: *I check system versus instance status checks to know if it is AWS hardware or our OS, read the console output and screenshot, then recover by stop/start, reboot or fixing the root volume on a rescue instance. In our case a monitoring server hung from memory and disk pressure; I rebooted it, verified the services, then tuned PHP-FPM, cleaned the journal and added alarms.*

Q66. An S3 bucket used for artifact storage was accidentally made public. What is your fix?

Also asked as: An S3 bucket was made public by mistake - how do you secure and audit it?

In simple words: The warehouse door was left open. Shut it immediately, check the CCTV for who came in, then fit a lock that cannot be left open again.

Detailed answer

Contain (minutes):

1. Turn on **Block Public Access** on the bucket (and account).
2. Remove the public bucket policy statement / public ACLs; set Object Ownership to BucketOwnerEnforced.

Assess:

3. **CloudTrail:** who changed the policy (`PutBucketPolicy`, `PutBucketAcl`, `PutPublicAccessBlock`) and when.
4. **S3 server access logs / CloudTrail data events:** which objects were downloaded by unknown IPs during the exposure window.
5. Check whether artifacts contained **secrets** (config files, keys). If yes, **rotate them all** and treat as a security incident.
6. Verify artifact integrity (checksums/signatures) in case someone **wrote** to it.
7. **Amazon Macie** scan of the bucket to find sensitive data (PII, credentials) that may have been exposed.
8. **IAM Access Analyzer for S3** to list any other buckets shared publicly or with outside accounts.

Prevent:

- **Account-level Block Public Access**, protected by an **SCP** that denies turning it off; AWS Config rule `s3-bucket-public-read-prohibited` with auto-remediation; IAM Access Analyzer alerts; bucket managed in Terraform so drift is visible; artifacts served via pre-signed URLs or VPC endpoint only.

Example:

```
aws s3api put-public-access-block --bucket build-artifacts \
  --public-access-block-configuration \
  BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
aws s3api delete-bucket-policy --bucket build-artifacts
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=EventName,AttributeValue=PutBucketPolicy --max-results 5
```

What to say to the interviewer: *I immediately enable Block Public Access and remove the public policy, then use CloudTrail to find who changed it and access logs to see what was downloaded,*

rotating any secrets that were in the artifacts. To prevent a repeat I enforce Block Public Access with an SCP, add a Config rule with auto-remediation and manage the bucket in Terraform.

Q67. NAT Gateway costs doubled in 24 hours during a live event, with no infrastructure changes. What could be causing it?

In simple words: The toll bridge bill doubled although you built no new roads. Something started driving over the bridge a lot more: find which car is making all the trips.

Detailed answer

NAT charges per GB processed, so something is sending much more data **through** it.

Likely silent causes:

- **Image pulls:** pods CrashLooping or nodes churning (Spot, autoscaling for the event) pulling large images from ECR/Docker Hub via NAT again and again.
- **S3/DynamoDB traffic via NAT** because there is no gateway endpoint (event traffic = more S3 reads/writes).
- **Logs and metrics shipping** to CloudWatch or a SaaS (debug logging enabled, traffic spike = more log volume).
- **Retry storms** to an external API or a dependency that is timing out.
- **Cross-AZ:** instances using a NAT in another AZ (extra inter-AZ charges).
- **Scaling itself:** more pods/nodes = more outbound calls (third-party CDN/DRM/analytics calls per viewer).
- Security: compromised instance exfiltrating data or mining crypto.

How to find it:

1. Cost Explorer: usage type `NatGateway-Bytes` by day/hour.
2. CloudWatch NAT metrics `BytesOutToDestination`, `BytesInFromDestination` per NAT.
3. **VPC Flow Logs** on the NAT ENI + Athena: top source IPs and destinations.

Fix: S3/DynamoDB gateway endpoints, ECR/CloudWatch interface endpoints, pull-through cache, fix crashloops, reduce log verbosity, NAT per AZ.

Example:

```
SELECT srcaddr, dstaddr, sum(bytes)/1024/1024/1024 AS gb
FROM vpc_flow_logs
WHERE interface_id = 'eni-0natgw' AND start > to_unixtime(now() - interval '24' hour)
GROUP BY srcaddr, dstaddr ORDER BY gb DESC LIMIT 20;
```

What to say to the interviewer: *NAT bills per GB, so I look for what started pushing more data through it, such as repeated image pulls from crashlooping pods or node churn, S3 traffic without a gateway endpoint, log shipping or retry storms. I confirm with NAT CloudWatch metrics and VPC Flow Logs in Athena, then fix with VPC endpoints, fixing the crashloops and reducing log volume.*

Q68. Your team sees a spike in cloud costs. How do you find out why and cut costs without hurting performance?

Also asked as: You suddenly see a 5x spike in cloud billing and the infra looks fine. Where do you look first, and how do you trace the bleed? Your monthly AWS bill increased unexpectedly - what tools and reports do you use? / Reduce cloud cost by 40% without impacting performance. Where do you start?

Tools and reports at a glance: Billing console Bills page (charges per service and region), Cost Explorer (filters, grouping, rightsizing recommendations), Cost Anomaly Detection, AWS Budgets, Cost and Usage Report (CUR) queried with Athena, Trusted Advisor cost checks, Compute Optimizer, and CloudTrail for who created what.

> **In simple words:** Your electricity bill jumped 5x. Check the meter room by room (service), then appliance by appliance (usage type, resource), and ask who plugged in something new. Then switch off the waste, not the fridge.

Detailed answer

Trace the bleed (top-down):

1. **Cost Explorer** with **hourly/daily granularity**: group by **service -> usage type -> resource**, and by linked account, region and tag. Find when it started.
2. **Cost Anomaly Detection** alerts usually point straight at the service and account.
3. **CUR (Cost and Usage Report) + Athena** for resource-level lines when Cost Explorer is not detailed enough.
4. **CloudTrail:** who created what (`RunInstances`, `CreateNatGateway`), especially in **regions you never use**. Compromised access keys launching **GPU crypto-mining instances** is a classic 5x cause: check all regions, disable the keys, terminate, open an AWS support case.

Usual silent culprits:

- **Data transfer:** NAT processing, cross-AZ traffic, internet egress.
- **Log ingestion:** CloudWatch `PutLogEvents` from someone leaving **debug logging** on.
- **S3 request costs:** a job doing millions of LIST/GET calls, or lifecycle transitions of tiny objects.
- **Lambda recursion loops** (Lambda writes to S3 which triggers the same Lambda); AWS now detects some loops, but check invocations.
- Forgotten large instances, unattached EBS, snapshots, idle load balancers, provisioned IOPS.

Cut without hurting performance: rightsizing from real p95 usage (Compute Optimizer), Savings Plans for baseline, Spot for stateless, Graviton, VPC endpoints, log levels and retention, S3 lifecycle/Intelligent-Tiering, schedule non-prod. Change one thing at a time and watch latency/error SLOs.

Prevent: AWS Budgets with alerts at 80/100 percent, anomaly detection subscriptions, tagging policy, SCPs denying unused regions.

Example:

```
aws ce get-cost-and-usage --time-period Start=2026-09-20,End=2026-09-24 \
  --granularity DAILY --metrics UnblendedCost \
  --group-by Type=DIMENSION,Key=SERVICE Type=DIMENSION,Key=USAGE_TYPE

for r in $(aws ec2 describe-regions --query "Regions[].RegionName" --output text); do
  echo "$r: $(aws ec2 describe-instances --region $r \
    --query 'length(Reservations[].Instances[])')"
done
```

A 40% reduction target, in order of impact:

1. **Visibility first:** cost allocation tags, Cost Explorer by service, account and tag, and the top 10 line items. Usually 3-4 items are most of the bill.
2. **Remove waste:** idle or unattached EBS volumes, old snapshots, unused Elastic IPs, idle load balancers, forgotten dev environments. Schedule non-prod to stop at night and on weekends (about 65% off those hours).
3. **Rightsize:** Compute Optimizer and CloudWatch data, over-provisioned instances and RDS, K8s requests vs actual usage, and move to Graviton where possible.
4. **Pricing models:** Savings Plans or RIs for the steady baseline, Spot for stateless, batch and CI workloads, and Karpenter consolidation.
5. **Data transfer and NAT:** VPC endpoints for S3 and ECR, keep traffic in the same AZ, and use CloudFront.
6. **Storage tiering:** S3 lifecycle to IA or Glacier, gp2 to gp3, and log retention limits.
7. **Guard performance:** agree SLOs (p95 latency, error rate) first, change in small steps, compare before and after, and keep budgets and anomaly alerts so costs don't creep back.

What to say to the interviewer: *I break the bill down in Cost Explorer by service, usage type and resource at hourly granularity, check Cost Anomaly Detection and CUR in Athena, and use CloudTrail to see who created what, including in unused regions where compromised keys may be mining crypto. Common silent culprits are NAT and data transfer, debug log ingestion, S3 requests and Lambda loops; I cut waste with rightsizing, Savings Plans, Spot and endpoints while watching SLOs, and prevent repeats with budgets and anomaly alerts.*

Q69. How would you handle a sudden traffic spike from 10K to 200K users?

In simple words: Twenty times more guests are coming. Serve snacks at the door (cache), open more counters (autoscale), queue orders instead of dropping them, and turn away only the extras politely if you must.

Detailed answer

If you know in advance (event, sale):

1. **Load test** at 1.5-2x the expected peak.
2. **Pre-scale:** scheduled scaling / raise min capacity, warm pools; pre-provision Lambda concurrency, DynamoDB capacity; check **service quotas** early (vCPU, Lambda concurrency, API Gateway).
3. For very sudden huge jumps, ALB scales automatically but you can use **LCU capacity reservation**; NLB and CloudFront handle bursts well.

Architecture for the spike:

- **CloudFront** caching for static and cacheable API responses (biggest win).
- **Stateless app tier** autoscaling on requests per target; EKS with Karpenter for fast nodes.
- **Database protection:** read replicas, ElastiCache, RDS Proxy; Aurora auto-scaling replicas.
- **Queues (SQS/Kafka)** to absorb writes asynchronously.
- **Rate limiting** (WAF, API Gateway) and **graceful degradation**: turn off non-critical features with feature flags.
- Dashboards and alarms on latency/errors; a war room during the event.

Example:

```
aws autoscaling put-scheduled-update-group-action --auto-scaling-group-name web-asg \
  --scheduled-action-name sale-prewarm --start-time 2026-10-02T12:30:00Z \
  --min-size 40 --max-size 200 --desired-capacity 60

User -> CloudFront (80% cache hit) -> WAF rate limit -> ALB -> ASG/EKS (autoscale)
                                                          -> ElastiCache -> RDS + replicas
                                                          -> SQS -> workers (async writes)
```

> ***What to say to the interviewer:*** *For a planned spike I load test, check quotas and pre-scale with scheduled actions, then rely on CloudFront caching, a stateless autoscaling tier, caches and read replicas to protect the database, and queues for writes. I add rate limiting and feature flags for graceful degradation and watch latency and error dashboards throughout.*

Q70. How do you design a fault-tolerant system that keeps running when one Availability Zone is down?

> **In simple words:** Keep three kitchens in three buildings, each able to handle the load if one burns down, and do not keep the only recipe book in one of them.

Detailed answer

- **Spread everything across 3 AZs:** ALB subnets, ASG/EKS nodes, ECS tasks, MSK brokers, ElastiCache replicas.
- **Static stability:** run enough capacity so 2 AZs can carry 100 percent of load without needing to launch anything during the failure (for example 3 AZs x 50 percent). Scaling during an AZ event may be slow or impossible.
- **Data:** RDS/Aurora Multi-AZ, S3 (regional, multi-AZ by default), DynamoDB (multi-AZ by default), EFS regional.
- **NAT Gateway per AZ** with per-AZ route tables.
- **Health checks** remove bad targets; ALB cross-zone load balancing on.
- **Kubernetes:** topology spread constraints across zones, PodDisruptionBudgets, zone-aware storage (EBS is zonal: use StatefulSets with a replica per AZ).
- **Avoid single-AZ dependencies:** one EC2 cron server, a single Redis node, a self-managed DB in one AZ.
- **Test:** use **ARC zonal shift** or FIS to simulate an AZ failure.

Example:

```
            ALB (AZ-a, AZ-b, AZ-c)
        /            |            \
  AZ-a: 3 nodes  AZ-b: 3 nodes  AZ-c: 3 nodes   (2 AZs alone = 100% load)
  NAT-a          NAT-b          NAT-c
  Aurora writer  Aurora reader  Aurora reader   (auto failover)

aws arc-zonal-shift start-zonal-shift --resource-identifier <alb-arn> \
  --away-from aps1-az1 --expires-in 1h --comment "AZ failure drill"
```

What to say to the interviewer: *I deploy every tier across three AZs with per-AZ NAT, Multi-AZ databases and topology spread for pods, and I keep enough capacity that two AZs can carry full load without scaling during the failure, which is static stability. I remove single-AZ dependencies and regularly test with ARC zonal shift or Fault Injection Service.*

Q71. You are asked to migrate workloads from on-premises to AWS with zero downtime. How would you approach it?

In simple words: Move house without ever leaving the family homeless: build the new house, keep both homes synced, move people one room at a time, and keep the old house ready in case the new one leaks.

Detailed answer

1. **Discover and plan:** inventory apps and dependencies (Migration Hub / Application Discovery Service), pick a strategy per app (rehost, replatform, refactor), define success criteria and rollback.
2. **Connectivity:** Site-to-Site VPN or Direct Connect, hybrid DNS with Route 53 Resolver.
3. **Build the landing zone** with IaC: accounts, VPCs, IAM, monitoring.
4. **Replicate continuously:**

- Servers: **AWS Application Migration Service (MGN)** block-level continuous replication.
- Databases: **DMS full load + CDC** (or native replication) so AWS stays in sync.
- Files: DataSync.

5. **Test** the AWS copy with non-disruptive test launches and load tests.
6. **Cutover gradually:** lower DNS TTL days before, shift traffic with Route 53 **weighted records** (1 -> 10 -> 50 -> 100 percent), keep the database single-writer at any moment.
7. **Rollback plan:** keep on-prem running and reverse replication until stable.
8. **Decommission** after a hypercare period.

Example:

```
Day -7 : Route 53 TTL 3600 -> 60
Day  0 : MGN + DMS CDC in sync, test instances pass
Day  1 : weight on-prem 90 / AWS 10  (read-only traffic first)
Day  3 : DB cutover in low traffic window (stop writes 1-2 min, CDC catch-up, switch)
Day  4 : weight 0 / 100, reverse replication to on-prem kept for rollback
Day 30 : decommission on-prem
```

What to say to the interviewer: *I set up hybrid connectivity and the AWS landing zone with IaC, then replicate continuously with Application Migration Service for servers and DMS CDC for databases. After testing, I shift traffic gradually using low-TTL weighted Route 53 records, keep the*

database single-writer during a short cutover, and keep on-prem running with reverse replication for rollback.

Q72. What strategies do you use for database migration with zero downtime?

Also asked as: How do you handle zero-downtime database migrations in a distributed application? How do you migrate an RDS database with minimal downtime?

In simple words: Build a new road next to the old one, let cars use both for a while, then close the old road only when everyone is on the new one.

Detailed answer

Moving to a new database/engine/version:

- **DMS full load + CDC:** copy existing data, then stream ongoing changes until cutover; validate with DMS data validation.
- **RDS Blue/Green Deployments:** RDS creates a synced green copy (for upgrades or parameter changes); switchover typically takes under a minute and keeps endpoint names.
- **Native replication** (logical replication for PostgreSQL, binlog replication for MySQL).
- Cutover: stop writes briefly, confirm lag = 0, switch the endpoint (RDS Proxy / DNS / secret), keep the old DB for rollback.
- A common wrong answer is "enable Multi-AZ". Multi-AZ gives high availability (a standby for failover) inside the same database; it does not move data to a new instance, engine, version or account, so it is not a migration method.

Schema changes in a live app (expand-contract):

1. **Expand:** add new column/table (nullable, no locks); deploy code writing to both old and new.
2. **Backfill** data in small batches.
3. Switch reads to the new structure.
4. **Contract:** remove the old column in a later release.

- Never rename or drop in the same release; use online tools (`gh-ost`, `pg_repack`, `CREATE INDEX CONCURRENTLY`).
- Migrations run in CI/CD as a separate, versioned step (Flyway/Liquibase) and must be backward-compatible.

Example:

```
aws rds create-blue-green-deployment --blue-green-deployment-name orders-pg16 \
  --source arn:aws:rds:ap-south-1:111111111111:db:orders-prod \
  --target-engine-version 16.4
aws rds switchover-blue-green-deployment \
  --blue-green-deployment-identifier bgd-abc123 --switchover-timeout 300

-- expand step (PostgreSQL)
ALTER TABLE users ADD COLUMN email_verified boolean;
CREATE INDEX CONCURRENTLY idx_users_email ON users(email);
```

What to say to the interviewer: *For moving databases I use DMS full load plus CDC or native replication, or RDS Blue/Green Deployments for upgrades, cutting over when lag is zero and keeping the old database for rollback. For schema changes I follow expand-contract with backward-compatible, versioned migrations so old and new app versions work at the same time.*

Q73. You must ship multi-region failover for live events in 2 weeks, and DNS-based routing is not allowed. What is your plan?

In simple words: Give users one permanent phone number that rings in two cities. If one office stops answering, the phone system rings the other one automatically, without printing a new phone book.

Detailed answer

Traffic layer (no DNS changes):

- **AWS Global Accelerator:** two static anycast IPs; endpoint groups in two regions (ALB/NLB) with health checks; failover in seconds; traffic dials to shift load.
- For HTTP content, **CloudFront origin groups** give origin failover (for GET/HEAD/OPTIONS requests).

2-week plan:

1. **Days 1-2:** agree RTO/RPO, scope only the critical live path (playback/API), pick the secondary region.
2. **Days 3-6:** deploy the same IaC stack in region 2 (warm standby, pre-scaled for the event); replicate images (ECR replication), secrets and config.
3. **Days 5-8:** data: Aurora Global Database / DynamoDB Global Tables / S3 CRR; for live streams, ingest to both regions.
4. **Days 7-9:** Global Accelerator with health checks, traffic dials 100/0.
5. **Days 10-12:** failover drills, load tests in region 2, runbook with clear decision owner.
6. **Days 13-14:** freeze, dashboards, war room.

Example:

```
Clients -> Global Accelerator (static IPs 75.2.x.x, 99.83.x.x)
             |-- endpoint group ap-south-1     dial 100%  (ALB, health check /healthz)
             +-- endpoint group ap-southeast-1 dial 0%    (warm, pre-scaled)

aws globalaccelerator update-endpoint-group --endpoint-group-arn <arn-primary> \
  --traffic-dial-percentage 0      # manual failover drill
```

What to say to the interviewer: *I would use Global Accelerator, whose static anycast IPs route to healthy regional endpoints and fail over in seconds without any DNS change, with CloudFront origin groups for cacheable content. In two weeks I would scope only the critical path, deploy a warm, pre-scaled copy via IaC in the second region, replicate data with global databases, and spend the final days on failover drills and a clear runbook.*

Q74. Application latency suddenly increased. The app runs in a private subnet behind a NAT Gateway. How do you investigate?

In simple words: If every trip out of the building goes through one exit gate, a jam at that gate slows everyone. First check whether the slowness is really on trips going out, then check the gate.

Detailed answer

1. **Is it outbound?** Traces (X-Ray/OTel) or app timing: is the slow span a call to an external API, AWS API or S3 through the NAT, or is it internal (DB, CPU)? If internal, NAT is not the issue.
2. **NAT Gateway CloudWatch metrics:**

- `ErrorPortAllocation` > 0: NAT ran out of source ports. A NAT supports about **55,000 simultaneous connections to each unique destination** (IP + port + protocol); fix by adding secondary private IPs to the NAT, spreading destinations, or reusing connections.
- `PacketsDropCount`: drops at the NAT.
- `ActiveConnectionCount`, `ConnectionAttemptCount` vs `ConnectionEstablishedCount`, `IdleTimeoutCount`.
- `BytesOutToDestination`: a sudden traffic jump (bandwidth scales up to 100 Gbps but bursts can queue).

3. **Cross-AZ NAT:** instances in AZ-b using a NAT in AZ-a add latency and cost; use one NAT per AZ with per-AZ route tables.
4. **Skip the NAT for AWS services:** S3/DynamoDB gateway endpoints, interface endpoints for STS, Secrets Manager, ECR, CloudWatch.
5. **Connection behaviour:** no keep-alive, a new TLS handshake per request, connection pool too small, idle connections dropped after 350 seconds of NAT idle timeout causing retries.
6. **The destination itself:** third-party API slow or rate limiting (429s), their region/latency, test from another network.
7. **DNS:** slow or failing lookups (Route 53 Resolver limits, missing caching).

Example:

```
aws cloudwatch get-metric-statistics --namespace AWS/NATGateway \
  --metric-name ErrorPortAllocation --dimensions Name=NatGatewayId,Value=nat-0abc \
  --start-time 2026-09-24T08:00:00Z --end-time 2026-09-24T10:00:00Z \
  --period 60 --statistics Sum

aws ec2 assign-private-nat-gateway-address --nat-gateway-id nat-0abc \
  --private-ip-address-count 2          # more IPs = more ports per destination
```

> ***What to say to the interviewer:*** *I first confirm with traces that the slow part is outbound traffic through the NAT, then check NAT metrics like ErrorPortAllocation for port exhaustion, PacketsDropCount and connection counts, plus cross-AZ NAT usage. Fixes include VPC endpoints so AWS traffic skips the NAT, connection reuse and keep-alive, extra NAT IPs, one NAT per AZ, and checking whether the third-party endpoint itself is slow.*

Q75. How would you expose a backend API securely through CloudFront and WAF, allowing only a certain IP range?

> **In simple words:** Put a guarded gate (CloudFront + WAF) in front with a guest list (IP set), and wall off the back door (origin) so the only way in is through that gate.

Detailed answer

Edge (CloudFront + WAF):

1. Create an **IP set** with the allowed CIDRs.
2. **Web ACL** (scope CLOUDFRONT, created in us-east-1) with **default action Block** and a rule **Allow if source IP in IP set**.
3. Add AWS **managed rule groups** (Core rule set, Known bad inputs, SQLi) and a **rate-based rule**, evaluated before the allow rule so allowed IPs still get inspected.
4. Attach the web ACL to the distribution; HTTPS only with an ACM certificate.

Lock the origin so nobody bypasses CloudFront:

- Best: **CloudFront VPC origins** pointing to an **internal ALB** in private subnets (no public exposure at all).
- Or public ALB with its security group allowing only the **CloudFront managed prefix list** (`com.amazonaws.global.cloudfront.origin-facing`), plus a **secret custom header** added by CloudFront and checked by an ALB listener rule (others get 403). Rotate the header value via Secrets Manager.
- For S3 origins use **OAC**.

Also: TLS end to end (CloudFront -> ALB on HTTPS), disable caching for the API or cache only safe GETs, forward only needed headers, and enable **WAF logs, CloudFront logs and ALB access logs**.

Example:

```
aws wafv2 create-ip-set --name partner-ips --scope CLOUDFRONT --region us-east-1 \
  --ip-address-version IPV4 --addresses 203.0.113.0/24 198.51.100.10/32

# ALB listener rule: forward only if the secret header matches, else fixed 403
aws elbv2 create-rule --listener-arn <listener-arn> --priority 1 \
  --conditions '[{"Field":"http-header","HttpHeaderConfig":
     {"HttpHeaderName":"X-Origin-Verify","Values":["<secret-value>"]}}]' \
  --actions Type=forward,TargetGroupArn=<tg-arn>
```

> ***What to say to the interviewer:*** *I attach a WAF web ACL to CloudFront with an IP set allow rule, a default block action, managed rule groups and rate limiting. Then I lock the origin using CloudFront VPC origins to a private ALB, or the CloudFront managed prefix list in the ALB security group plus a secret header checked by a listener rule, with TLS end to end and WAF, CloudFront and ALB logging.*

Q76. CloudWatch alerts are firing for Lambda timeout errors. How do you debug?

> **In simple words:** The worker keeps running out of time. Find out whether he is waiting for someone else (a slow call), stuck at a locked door (network), or just too slow for the job (not enough memory/CPU).

Detailed answer

1. **Compare `Duration` with the configured timeout:** are invocations hitting the limit exactly (hung) or slowly rising (growing load/data)?
2. **Logs Insights:** search `Task timed out after` and see which requests or inputs time out; look at the `REPORT` lines for init duration and memory used.
3. **X-Ray traces** show which downstream call (DB, HTTP API, S3) consumes the time.

4. **VPC Lambda without internet path:** a function in private subnets with no NAT or VPC endpoint **hangs** on AWS API calls (Secrets Manager, S3, SQS) until the timeout. Add endpoints or a NAT.
5. **Cold starts/init:** heavy init code, large packages; use provisioned concurrency or SnapStart where supported.
6. **Memory:** more memory also means more CPU; check `Max Memory Used` and try higher memory.
7. **Database connections:** exhausted connections make calls wait; use **RDS Proxy** and reuse connections outside the handler.
8. **SDK and HTTP client timeouts/retries** must be **shorter than the Lambda timeout**, or retries silently eat the whole budget.
9. **Throttles and concurrency:** `Throttles` metric, reserved concurrency, downstream rate limits.
10. **SQS triggers:** the queue **visibility timeout** should be at least 6x the function timeout, otherwise messages reappear and are processed twice.

Example:

```
fields @timestamp, @requestId, @message
| filter @message like /Task timed out/
| stats count(*) as timeouts by bin(5m)

fields @timestamp, @duration, @maxMemoryUsed, @initDuration
| filter @type = "REPORT"
| sort @duration desc | limit 20
```

> ***What to say to the interviewer:*** *I compare Duration against the timeout, search Logs Insights for Task timed out and use X-Ray to find the slow downstream call. Common causes are a VPC Lambda with no NAT or endpoint hanging on AWS APIs, cold starts, too little memory, exhausted DB connections, SDK timeouts longer than the Lambda timeout, throttling, and an SQS visibility timeout that is too short.*

Q77. You want to auto-remediate high CPU on EC2 by rebooting it through Lambda. What is the logic?

> **In simple words:** Rebooting is like turning a laptop off and on: it often works, but you learn nothing about why it was slow. Take a quick photo of the problem first, and never let the robot reboot the same machine in an endless loop.

Detailed answer

First, you may not need Lambda: a CloudWatch alarm can reboot natively using the EC2 **reboot alarm action** (`arn:aws:automate:<region>:ec2:reboot`).

If you need logic (allowlists, diagnostics, notifications), the Lambda flow is:

1. CloudWatch alarm (CPU > 95 percent for 10 minutes) -> **EventBridge** "Alarm State Change" rule (or SNS) -> Lambda.
2. **Parse the instance ID** from the alarm dimensions.

3. **Guardrails:** only act on instances tagged `auto-remediate=true`; skip databases and stateful nodes; enforce a **cooldown** (for example no second reboot within 30 minutes, tracked in a tag or DynamoDB) to avoid reboot loops.
4. **Capture diagnostics** first with **SSM Run Command** (`top`, `ps`, logs) saved to S3.
5. Call **RebootInstances**.
6. **Notify** Teams/Slack with instance, time and diagnostics link; escalate if it happens again.
7. **IAM least privilege:** `ec2:RebootInstances` only on tagged instances, `ssm:SendCommand`, `ec2:CreateTags`.

Warning: rebooting **hides the root cause**. Use it as a temporary safety net and still do RCA (memory leak, runaway cron, traffic).

Example (the decision steps as CLI):

```
ID=i-0abc123def4567890
TAG=$(aws ec2 describe-tags --filters Name=resource-id,Values=$ID \
  Name=key,Values=auto-remediate --query "Tags[0].Value" --output text)
[ "$TAG" != "true" ] && echo "not allowlisted, notify only" && exit 0

LAST=$(aws ec2 describe-tags --filters Name=resource-id,Values=$ID \
  Name=key,Values=last-auto-reboot --query "Tags[0].Value" --output text)
# if LAST is within 30 minutes -> skip reboot and escalate to on-call

aws ssm send-command --instance-ids $ID --document-name AWS-RunShellScript \
  --parameters 'commands=["top -b -n1 | head -30","ps aux --sort=-%cpu | head"]' \
  --output-s3-bucket-name ops-diagnostics
aws ec2 reboot-instances --instance-ids $ID
aws ec2 create-tags --resources $ID --tags Key=last-auto-reboot,Value=$(date -u +%s)
```

> ***What to say to the interviewer:*** *If a plain reboot is enough, a CloudWatch alarm can reboot the instance natively without Lambda. With Lambda, the alarm goes through EventBridge, the function checks an allowlist tag and a cooldown to avoid reboot loops, captures diagnostics with SSM Run Command, reboots, and notifies the team, while I still do the RCA because a reboot only hides the cause.*

Q78. An automation job that starts and stops EMR clusters suddenly fails with IAM errors. How do you debug and fix it?

> **In simple words:** The robot's key stopped opening the door. Read exactly which door refused it, check which key the robot is actually carrying, and see who changed the lock.

Detailed answer

1. **Read the exact error:** which **action** and **resource** were denied, and whether the message says **explicit deny**, **service control policy** or **permissions boundary** (newer AccessDenied messages include this).
2. **Who am I?** `aws sts get-caller-identity` from the job's environment. The job may now run as a different role (profile changed, instance role replaced, expired or rotated keys, assume-role failing).
3. **CloudTrail:** filter by `errorCode = AccessDenied` / `UnauthorizedOperation` for the exact event and principal.
4. **Encoded messages:** decode with `aws sts decode-authorization-message`.

5. **IAM Policy Simulator** for the role against `elasticmapreduce:RunJobFlow`, `TerminateJobFlows`, and so on.
6. **Classic EMR gap:** `iam:PassRole`**.** Starting a cluster passes the EMR service role and the EC2 instance profile role; the caller needs `iam:PassRole` on those role ARNs.
7. **What changed?** CloudTrail for `PutRolePolicy`, `DetachRolePolicy`, SCP updates, new permission boundaries, tag-based conditions (cluster tags missing), KMS key policy changes for encrypted clusters.
8. **Fix** with the smallest least-privilege change, test with the simulator, commit it to IaC so it does not drift again, and add an alarm on repeated failures.

Example:

```
aws sts get-caller-identity
aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::111111111111:role/emr-scheduler \
  --action-names elasticmapreduce:RunJobFlow iam:PassRole \
  --resource-arns "*"
```

```
{
  "Effect": "Allow",
  "Action": "iam:PassRole",
  "Resource": [
    "arn:aws:iam::111111111111:role/EMR_DefaultRole",
    "arn:aws:iam::111111111111:role/EMR_EC2_DefaultRole"
  ]
}
```

> ***What to say to the interviewer:*** *I read the exact denied action, resource and whether an explicit deny, SCP or boundary is involved, confirm the job's identity with sts get-caller-identity, and find the event in CloudTrail and test in the Policy Simulator. For EMR the usual gap is iam:PassRole on the service and instance-profile roles; I then check what changed, apply a least-privilege fix and commit it to IaC.*

Q79. An EMR cluster runs 24x7 but is used only 4 hours a day. What automation do you implement?

> **In simple words:** Do not keep a bus running all day for a 4-hour route. Call a bus when passengers arrive and send it home when the trip ends.

Detailed answer

1. **Transient clusters:** launch a cluster per job run, submit steps, and **auto-terminate after the last step**. Orchestrate with **Step Functions**, Airflow (MWAA), or **EventBridge Scheduler**.
2. **Auto-termination policy:** for interactive clusters, terminate after an **idle timeout** (for example 1 hour).
3. **Keep data in S3** (EMRFS), not HDFS, so clusters are disposable; keep the Hive/Glue Data Catalog external.
4. **EMR managed scaling:** set min/max units so the cluster shrinks when idle.
5. **Spot for task nodes** (and instance fleets with several instance types); On-Demand only for the primary/core nodes.
6. **EMR Serverless:** pay only while the job runs; no cluster to manage at all.

7. **Graviton instances** and right-sizing; tag and track cost per job.

Savings: roughly 20 hours a day of idle cost (about 80 percent) disappear, before Spot savings.

Example:

```
aws emr create-cluster --name nightly-etl --release-label emr-7.2.0 \
  --applications Name=Spark --use-default-roles \
  --instance-groups InstanceGroupType=MASTER,InstanceCount=1,InstanceType=m7g.xlarge \
    InstanceGroupType=CORE,InstanceCount=2,InstanceType=r7g.2xlarge \
    InstanceGroupType=TASK,InstanceCount=4,InstanceType=r7g.2xlarge,BidPrice=OnDemandPrice \
  --steps Type=Spark,Name=etl,ActionOnFailure=TERMINATE_CLUSTER,Args=[s3://jobs/etl.jar] \
  --auto-terminate

aws emr put-auto-termination-policy --cluster-id j-ABC123 \
  --auto-termination-policy IdleTimeout=3600
```

> ***What to say to the interviewer:*** *I would switch to transient clusters started per job by Step Functions or EventBridge Scheduler that auto-terminate after their steps, keep data in S3 so clusters are disposable, and add an idle auto-termination policy for any interactive cluster. Managed scaling, Spot task nodes and possibly EMR Serverless cut cost further, removing about 80 percent of the idle spend.*

Q80. How do you identify underutilized EC2 instances across multiple AWS accounts?

> **In simple words:** Walk through every office in the company with one checklist and find the computers that are switched on but nobody really uses.

Detailed answer

Find them (organization-wide):

1. **AWS Compute Optimizer** enabled at the **organization level** (management or delegated admin account): rightsizing recommendations for EC2, ASGs, EBS, Lambda across all accounts. Install the **CloudWatch agent** so it also uses memory metrics.
2. **Cost Explorer rightsizing recommendations** (from the management account) show savings in money.
3. **Trusted Advisor** "Low Utilization Amazon EC2 Instances" check (needs Business support or higher), viewable across the org.
4. **CloudWatch cross-account observability:** one monitoring account with dashboards of CPU, network and memory for all accounts (for example p95 CPU < 10 percent for 14 days).
5. **Tag hygiene:** owner, env and cost-center tags so each finding goes to the right team; untagged resources are reported.

Act:

- **Rightsize** (smaller type or Graviton), **stop or schedule** idle dev/test instances, terminate orphans after owner confirmation.
- Move steady baseline to **Savings Plans** after rightsizing (not before).
- Repeat monthly; track savings.

Example:

```
aws compute-optimizer update-enrollment-status --status Active --include-member-accounts

aws compute-optimizer get-ec2-instance-recommendations \
  --filters name=Finding,values=Overprovisioned \
  --query "instanceRecommendations[].{acct:accountId,id:instanceArn,now:currentInstanceType,
  rec:recommendationOptions[0].instanceType}" --output table
```

What to say to the interviewer: *I enable Compute Optimizer at the organization level with the CloudWatch agent for memory data, and combine it with Cost Explorer rightsizing, Trusted Advisor low-utilization checks and cross-account CloudWatch dashboards. Using owner tags, teams then rightsize, schedule or terminate the instances, and only after rightsizing do we buy Savings Plans for the baseline.*

Q81. Some APIs behind API Gateway are unsecured. How would you secure them?

In simple words: Some doors of the building have no lock. First walk around and list every unlocked door, then fit the right lock on each (ID check, badge, guest list), put a guard at the entrance, and install cameras.

Detailed answer

1. **Inventory:** list every route/method and find the ones with `authorizationType = NONE`. Also check for backend URLs (ALB, Lambda function URLs) reachable directly, bypassing API Gateway.
2. **Authentication per caller type:**

- End users: **Cognito user pool authorizer** (REST) or **JWT authorizer** (HTTP API) with any OIDC provider.
- Custom logic (tokens, headers): **Lambda authorizer**, with result caching.
- Service-to-service: **IAM auth** (`AWS_IAM`), callers sign with **SigV4** using their role.

3. **API keys + usage plans:** for throttling, quotas and metering per client. They are **not security** (keys are easy to leak); always pair them with a real authorizer.
4. **Resource policies:** allow only certain IP ranges, accounts or VPC endpoints (`aws:SourceIp`, `aws:SourceVpce`); use **private APIs** for internal-only traffic.
5. **AWS WAF** on the stage: managed rule groups (common, SQLi, known bad inputs) and **rate-based rules**.
6. **mTLS on a custom domain** for partners/B2B: clients must present a certificate from your truststore; disable the default `execute-api` endpoint so the custom domain is the only way in.
7. **Throttling** per stage and per method to protect backends.
8. **Request validation** (models/schemas for body, required params and headers) to reject bad input early.
9. **TLS 1.2+** security policy on custom domains.
10. **Logging and alerting:** CloudWatch access logs (caller, IP, status, latency), execution logs while debugging, alarms on 4XX/5XX spikes, CloudTrail for config changes.
11. **Do not expose backends directly:** internal ALB via VPC link, Lambda permission only for API Gateway, and IaC so the fix does not drift back.

Example:

```
# Find unauthenticated methods (REST API)
aws apigateway get-resources --rest-api-id a1b2c3 --embed methods \
  --query "items[].{path:path,methods:resourceMethods}" --output json \
  | grep -B3 '"authorizationType": "NONE"'

# Attach a Cognito authorizer to a method
aws apigateway update-method --rest-api-id a1b2c3 --resource-id r1s2t3 \
  --http-method GET --patch-operations \
  op=replace,path=/authorizationType,value=COGNITO_USER_POOLS \
  op=replace,path=/authorizerId,value=auth123
aws apigateway create-deployment --rest-api-id a1b2c3 --stage-name prod
```

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow", "Principal": "*", "Action": "execute-api:Invoke",
      "Resource": "execute-api:/*" },
    { "Effect": "Deny", "Principal": "*", "Action": "execute-api:Invoke",
      "Resource": "execute-api:/*",
      "Condition": { "NotIpAddress": { "aws:SourceIp": ["203.0.113.0/24"] } } }
  ]
}
```

What to say to the interviewer: *I first inventory every route with authorizationType NONE and any backend reachable directly, then add the right authentication: Cognito or JWT authorizers for users, Lambda authorizers for custom tokens and IAM SigV4 for service-to-service calls. I layer resource policies or private APIs, WAF managed and rate-based rules, mTLS for partners, throttling, request validation and TLS 1.2, and log and alarm on access; API keys and usage plans are only for metering, not security.*